COL334 Assignment 2 — The Socket Exchange

Experiment Report

Team: Avvaru Yasasvi (2024CS10063), Hardhik Reddy (2024CS10466) **Language / build:** C++17, built with the system `c++` (FreeBSD base clang) via `make`. **Environment:** FreeBSD 14.4-RELEASE (amd64) in VirtualBox, 4 GB RAM / 20 GB disk.

Build and setup

```
make
./server/run-server 127.0.0.1 5000
```

`make` builds `exchange_server`, `trader_client`, and `market_data_client`. The server prints its listening address and the event loop it was built with (`poll` by default).

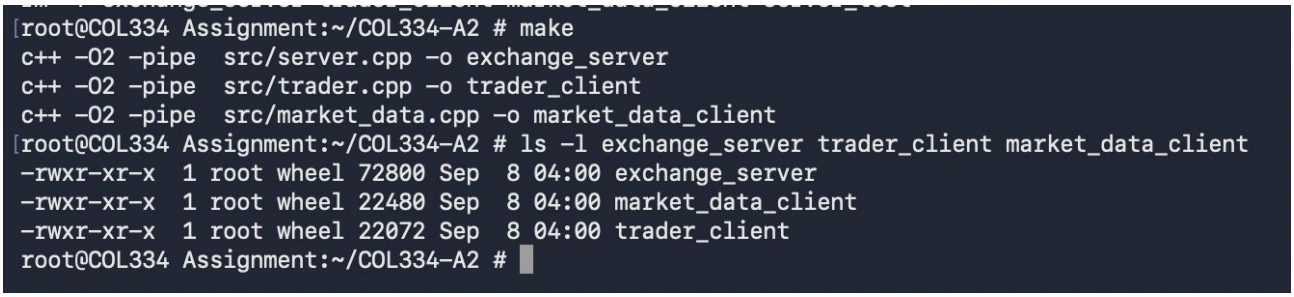

Figure 1: `make` builds the three binaries; `ls -l` on them

Implementation Decisions

Concurrency / I/O mechanism

The Exchange Server is a **single process with a single thread running one event-driven readiness loop**. Every socket — the listening socket and every client connection — is set non-blocking. The loop:

1. calls a readiness primitive that blocks until at least one socket needs attention;
2. `accept()`s all pending connections if the listening socket is ready;
3. for each ready client, `recv()`s available bytes into a per-connection input buffer, splits it on `\n`, and processes each complete message;
4. `send()`s as much of each client's queued output as the kernel accepts, keeping the remainder for later.

The readiness primitive is selected at build time through a small abstraction (`src/event_poller.hpp`):

Build	Primitive	Purpose
`make` (default)	`poll(2)`	portable; used for grading and all experiments below
`make POLLER=kqueue`	`kqueue`	O(ready) wakeups; used for the §6.9 scalability bonus
`make POLLER=epoll`	`epoll`	Linux-only, for cross-checking on a Linux host

Why this design

- **Isolation.** The assignment requires that one idle, slow, or abruptly-disconnecting client never disturbs the others (§4.3, §4.4). With a single `poll()` loop and non-blocking sockets the server never blocks inside `recv()` or `send()` for any one client — it only ever blocks in the readiness wait, which watches *all* clients simultaneously.
- **Cost.** No per-connection thread/process means no per-connection stack or scheduler overhead: application state is ~0.3 KB per connection, and the server holds 70 000 idle connections in ~26 MB RSS on the default `poll` build, ~44 MB on the `kqueue` build (measured, §6.9). A thread-per-connection design would hit thread-stack and lock-contention limits well before the 70 000-connection bonus target, for a workload that is I/O-bound rather than CPU-bound.
- **Observability.** A single explicit loop makes the underlying TCP behaviour — byte-stream framing, partial reads/writes, backpressure, FIN/RST detection — directly visible, which is the purpose of the experiments.

Other decisions relevant to networking

- **Message framing.** Per-connection byte buffer, split on `\n` (a trailing `\r` is tolerated). A read cursor is advanced instead of erasing from the front, so re-framing is linear, not quadratic. A 64 KiB per-connection input cap is a safety valve for out-of-spec input; hitting it closes only that one connection.
- **Output queueing.** One contiguous per-connection output buffer plus a write offset. On `EAGAIN` the unsent tail is kept and write-interest is registered with the readiness loop; it drains on a later iteration. An 8 MiB cap bounds a non-reading client (§ Experiment 7); exceeding it drops that one connection.
- **Order book.** Per `(instrument, side)`: `std::map<price, list<Order>>`, FIFO within a price level, plus `unordered_map<order_id, iterator>` for O(1) `CANCEL`. Matching occurs only at an exactly equal price (§2.6).
- **Roles.** A connection is unassigned until its first command: `LOGIN` ⇒ Trader, `SUBSCRIBE/UNSUBSCRIBE` ⇒ Market-Data. A command not permitted for the established role is answered with `ERROR` and ignored (§4.5).
- **Connection lifetime.** A resting order is keyed to the submitting connection, not the username. It survives that client's disconnect and can still match later; no `BOUGHT/SOLD` is sent to the departed client, but subscribers still receive the `TRADE` (§2.6, confirmed on Piazza).
- **Scalability.** At startup the server raises its own `RLIMIT_NOFILE` soft limit to the hard limit, does **not** pin `SO_SNDBUF/SO_RCVBUF` (which would reserve tens of GiB across 70 000 sockets), and, when the descriptor ceiling is reached, rejects new connections without crashing or busy-spinning.

Experiment 1 — Listening and Connected Sockets

Run: `python3 experiment.py 1`

Question. After the client has connected, identify the TCP sockets associated with the Exchange Server. How does the listening socket differ from the socket representing the connection to the client?

Approach. Start the experiment (server + one idle client). In a second terminal, list the server's sockets and the server process's open file descriptors, and read the TCP state and endpoint columns.

Commands / tools.

```
sockstat -4 | grep ':5000'
netstat -an -p tcp | grep '\.5000'
procstat -f $(pgrep exchange_server)
```

Observation. With the experiment client connected, `sockstat` showed the server process (`exchange_s`, PID 6674) holding two sockets, and the client (`python3`, PID 6673) holding one:

```
USER  COMMAND     PID   FD PROTO LOCAL ADDRESS      FOREIGN ADDRESS
root  exchange_s  6674  4  tcp4  127.0.0.1:5000     *:*
root  exchange_s  6674  5  tcp4  127.0.0.1:5000     127.0.0.1:51829
```

```
root  python3.12  6673  3  tcp4  127.0.0.1:51829    127.0.0.1:5000
```

`netstat -an` showed the matching TCP states:

```
127.0.0.1.5000    127.0.0.1.51829   ESTABLISHED
127.0.0.1.51829   127.0.0.1.5000    ESTABLISHED
127.0.0.1.5000    *.*               LISTEN
```

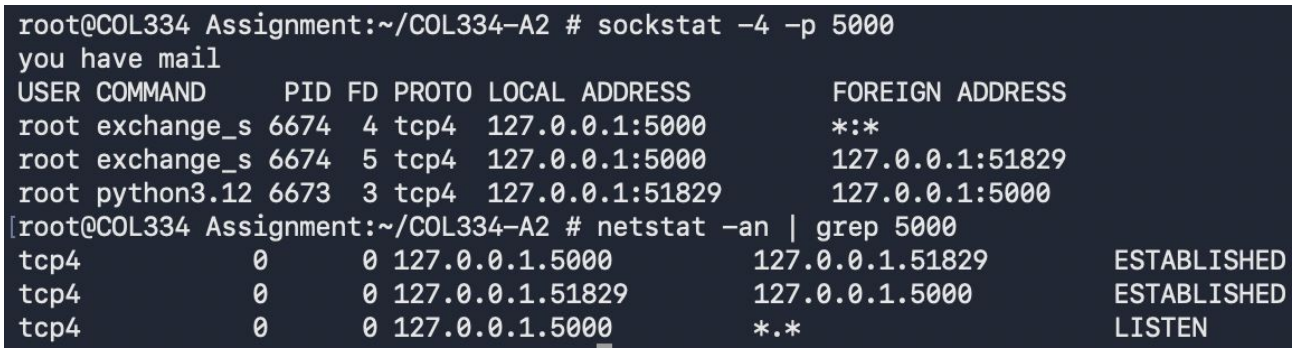

Figure 2: `sockstat -4 -p 5000` and `netstat -an | grep 5000` — server FD 4 (LISTEN) and FD 5 (ESTABLISHED to :51829)

Answer. The server has two kinds of TCP socket. The **listening socket** (server FD 4, from `socket()` + `bind()` + `listen()`) is in state `LISTEN`, is bound to `127.0.0.1:5000`, and has no foreign address (`*:*`). It carries no data; it only produces new connections through `accept()`. The **connected socket** (server FD 5, returned by `accept()`) is in state `ESTABLISHED` and has a full four-part address: local `127.0.0.1:5000` and the specific remote `127.0.0.1:51829` (the client's end, its FD 3). Application data flows only on the connected socket. The two are separate file descriptors, and because the listening socket stays in `LISTEN` the server can still accept more clients while talking to this one.

Experiment 2 — Observing TCP Connection States

Run: `python3 experiment.py 2`

Question. How does the TCP state of the client–server connection change during its lifetime, and what events cause the observed state changes?

Approach. Observe the connection at three points the harness pauses for: (1) while it is being established, (2) while established and idle, (3) after the client closes it. Capture packets throughout to correlate states with segments.

Commands / tools.

```
tcpdump -i lo0 -n 'tcp port 5000'          # in a terminal before the run
netstat -an -p tcp | grep '\.5000'         # repeat at each phase
sockstat -4 | grep ':5000'
```

Observation. During phase 1 (client connected and idle), `netstat` showed the connection `ESTABLISHED` at both ends, with the server still `LISTEN`ing (client ephemeral port 16831):

```
127.0.0.1.5000    127.0.0.1.16831   ESTABLISHED
127.0.0.1.16831   127.0.0.1.5000    ESTABLISHED
127.0.0.1.5000    *.*               LISTEN
```

After the client closed, the `ESTABLISHED` rows were gone and only the listening socket remained:

```
127.0.0.1.5000    *.*               LISTEN
```

We did not catch an intermediate `CLOSE_WAIT` or `TIME_WAIT` row; by the time the periodic `netstat` ran, the connection had already left the table.

```
root@COL334 Assignment:~/COL334-A2 # netstat -an -p tcp | grep 5000
tcp4           0      0 127.0.0.1.5000         127.0.0.1.16831        ESTABLISHED
tcp4           0      0 127.0.0.1.16831        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         *.*                    LISTEN
root@COL334 Assignment:~/COL334-A2 # netstat -an -p tcp | grep 5000
tcp4           0      0 127.0.0.1.5000         *.*                    LISTEN
```

Figure 3: `netstat` phase 1 (top): two `ESTABLISHED` rows for :16831 plus `LISTEN`. After the client closed (bottom): only the `LISTEN` row remains

Answer. The connection reaches `ESTABLISHED` after the three-way handshake (SYN, SYN-ACK, ACK) and stays there, with no packets, while idle. When the client calls `close()` it sends a FIN. On the server side `recv()` returns 0, the server closes that socket, and the connection is torn down with the normal FIN/ACK exchange from both sides. The transient states on the way down (`CLOSE_WAIT` on the server, `FIN_WAIT_2` and `TIME_WAIT` on the client that closed first) last only milliseconds here, so our periodic `netstat` did not capture them; what we can show is `ESTABLISHED` before the close and the connection gone afterwards. The listening socket stayed in `LISTEN` the whole time, so closing one client did not affect the server.

Experiment 3 — TCP as a Byte Stream

Run: `python3 experiment.py 3`

Question. Does the Exchange Server receive the application-level message as one complete unit, or can it be received in multiple pieces? Explain what this demonstrates about the relationship between TCP and application-level message boundaries.

Approach. The harness sends the single message `LOGIN experiment_trader\n` as four separate `send()` calls (`"LOGIN "`, `"experiment"`, `"_trader"`, `"\n"`) with pauses between them. Watch the segments on the wire and the server's `recv()` syscalls, and confirm the server only replies `OK` after the final `\n`.

Commands / tools.

```
tcpdump -i lo0 -n -X 'tcp port 5000'
truss -f -p $(pgrep exchange_server)        # or: ktrace -p <pid> ; kdump
```

Observation. `tcpdump` showed the four writes arriving as four separate data-carrying segments, each pushed about 0.2 s after the previous one (which matches the delay the harness inserts):

Time	TCP seq	Length	Bytes	Text
04:15:06.084075	1:7	6	`4c 4f 47 49 4e 20`	`LOGIN`
04:15:06.287146	7:17	10	`65 78 70 65 72 69 6d 65 6e 74`	`experiment`
04:15:06.488136	17:24	7	`5f 74 72 61 64 65 72`	`_trader`
04:15:06.688994	24:25	1	`0a`	`\n`

About 1.4 ms after the newline arrived, the server sent its reply:

04:15:06.690381 | server → client | 3 | `4f 4b 0a` | `OK\n` |

The kernel reported 0 packets dropped.

```
[root@COL334 Assignment:~/COL334-A2 # tcpdump -i lo0 -n -X port 5000                    ]
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on lo0, link-type NULL (BSD loopback), snapshot length 262144 bytes
04:15:05.983083 IP 127.0.0.1.25115 > 127.0.0.1.5000: Flags [S], seq 3052794835, win 65535, opt
ions [mss 16344,nop,wscale 8,sackOK,TS val 2452442583 ecr 0], length 0
        0x0000:  4500 003c 0000 4000 4006 0000 7f00 0001  E..<..@.@.......
        0x0010:  7f00 0001 621b 1388 b5f5 f3d3 0000 0000  ....b...........
        0x0020:  a002 ffff fe30 0000 0204 3fd8 0103 0308  .....0....?.....
        0x0030:  0402 080a 922d 4dd7 0000 0000            .....-M.....
04:15:05.983141 IP 127.0.0.1.5000 > 127.0.0.1.25115: Flags [R.], seq 0, ack 3052794836, win 0,
 length 0
        0x0000:  4500 0028 0000 4000 4006 0000 7f00 0001  E..(..@.@.......
        0x0010:  7f00 0001 1388 621b 0000 0000 b5f5 f3d4  ......b.........
        0x0020:  5014 0000 fe1c 0000                      P.......
04:15:06.083558 IP 127.0.0.1.57255 > 127.0.0.1.5000: Flags [S], seq 3697457060, win 65535, opt
ions [mss 16344,nop,wscale 8,sackOK,TS val 4282593921 ecr 0], length 0
        0x0000:  4500 003c 0000 4000 4006 0000 7f00 0001  E..<..@.@.......
        0x0010:  7f00 0001 dfa7 1388 dc62 b7a4 0000 0000  .........b......
        0x0020:  a002 ffff fe30 0000 0204 3fd8 0103 0308  .....0....?.....
        0x0030:  0402 080a ff43 3281 0000 0000            .....C2.....
04:15:06.083686 IP 127.0.0.1.5000 > 127.0.0.1.57255: Flags [S.], seq 3950560789, ack 369745706
1, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 1544652804 ecr 4282593921], length
 0
        0x0000:  4500 003c 0000 4000 4006 0000 7f00 0001  E..<..@.@.......
        0x0010:  7f00 0001 1388 dfa7 eb78 c615 dc62 b7a5  .........x...b..
        0x0020:  a012 ffff fe30 0000 0204 3fd8 0103 0308  .....0....?.....
        0x0030:  0402 080a 5c11 8804 ff43 3281            ....\....C2.
04:15:06.083704 IP 127.0.0.1.57255 > 127.0.0.1.5000: Flags [.], ack 1, win 320, options [nop,n
op,TS val 4282593921 ecr 1544652804], length 0
        0x0000:  4500 0034 0000 4000 4006 0000 7f00 0001  E..4..@.@.......
        0x0010:  7f00 0001 dfa7 1388 dc62 b7a5 eb78 c616  .........b...x..
        0x0020:  8010 0140 fe28 0000 0101 080a ff43 3281  ...@.(.......C2.
        0x0030:  5c11 8804                                \...
04:15:06.083784 IP 127.0.0.1.57255 > 127.0.0.1.5000: Flags [R.], seq 1, ack 1, win 0, options
[nop,nop,TS val 4282593921 ecr 1544652804], length 0
        0x0000:  4500 0034 0000 4000 4006 0000 7f00 0001  E..4..@.@.......
        0x0010:  7f00 0001 dfa7 1388 dc62 b7a5 eb78 c616  .........b...x..
        0x0020:  8014 0000 fe28 0000 0101 080a ff43 3281  .....(.......C2.
        0x0030:  5c11 8804                                \...
04:15:06.083904 IP 127.0.0.1.49900 > 127.0.0.1.5000: Flags [S], seq 1411595990, win 65535, opt
ions [mss 16344,nop,wscale 8,sackOK,TS val 3362177611 ecr 0], length 0
        0x0000:  4500 003c 0000 4000 4006 0000 7f00 0001  E..<..@.@.......
        0x0010:  7f00 0001 c2ec 1388 5423 3ed6 0000 0000  ........T#>.....
        0x0020:  a002 ffff fe30 0000 0204 3fd8 0103 0308  .....0....?.....
        0x0030:  0402 080a c866 c24b 0000 0000            .....f.K....
04:15:06.083939 IP 127.0.0.1.5000 > 127.0.0.1.49900: Flags [S.], seq 4203138308, ack 141159599
1, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 761314306 ecr 3362177611], length
0
        0x0000:  4500 003c 0000 4000 4006 0000 7f00 0001  E..<..@.@.......
        0x0010:  7f00 0001 1388 c2ec fa86 cd04 5423 3ed7  ............T#>.
        0x0020:  a012 ffff fe30 0000 0204 3fd8 0103 0308  .....0....?.....
        0x0030:  0402 080a 2d60 bc02 c866 c24b            ....-`...f.K
```

Figure 4: `tcpdump -X` — connection setup and the start of the four writes

```
        0x0030:  2d60 bc02                                  -`..
04:15:06.084075 IP 127.0.0.1.49900 > 127.0.0.1.5000: Flags [P.], seq 1:7, ack 1, win 320, opti
ons [nop,nop,TS val 3362177611 ecr 761314306], length 6
        0x0000:  4500 003a 0000 4000 4006 0000 7f00 0001  E..:..@.@.......
        0x0010:  7f00 0001 c2ec 1388 5423 3ed7 fa86 cd05  ........T#>.....
        0x0020:  8018 0140 fe2e 0000 0101 080a c866 c24b  ...@.........f.K
        0x0030:  2d60 bc02 4c4f 4749 4e20                 -`..LOGIN.
04:15:06.120819 IP 127.0.0.1.5000 > 127.0.0.1.49900: Flags [.], ack 7, win 320, options [nop,n
op,TS val 761314343 ecr 3362177611], length 0
        0x0000:  4500 0034 0000 4000 4006 0000 7f00 0001  E..4..@.@.......
        0x0010:  7f00 0001 1388 c2ec fa86 cd05 5423 3edd  ............T#>.
        0x0020:  8010 0140 fe28 0000 0101 080a 2d60 bc27  ...@.(......-`.'
        0x0030:  c866 c24b                                .f.K
04:15:06.287146 IP 127.0.0.1.49900 > 127.0.0.1.5000: Flags [P.], seq 7:17, ack 1, win 320, opt
ions [nop,nop,TS val 3362177815 ecr 761314343], length 10
        0x0000:  4500 003e 0000 4000 4006 0000 7f00 0001  E..>..@.@.......
        0x0010:  7f00 0001 c2ec 1388 5423 3edd fa86 cd05  ........T#>.....
        0x0020:  8018 0140 fe32 0000 0101 080a c866 c317  ...@.2.......f..
        0x0030:  2d60 bc27 6578 7065 7269 6d65 6e74       -`.'experiment
04:15:06.319461 IP 127.0.0.1.5000 > 127.0.0.1.49900: Flags [.], ack 17, win 320, options [nop,
nop,TS val 761314542 ecr 3362177815], length 0
        0x0000:  4500 0034 0000 4000 4006 0000 7f00 0001  E..4..@.@.......
        0x0010:  7f00 0001 1388 c2ec fa86 cd05 5423 3ee7  ............T#>.
        0x0020:  8010 0140 fe28 0000 0101 080a 2d60 bcee  ...@.(......-`..
        0x0030:  c866 c317                                .f..
04:15:06.488136 IP 127.0.0.1.49900 > 127.0.0.1.5000: Flags [P.], seq 17:24, ack 1, win 320, op
tions [nop,nop,TS val 3362178012 ecr 761314542], length 7
        0x0000:  4500 003b 0000 4000 4006 0000 7f00 0001  E..;..@.@.......
        0x0010:  7f00 0001 c2ec 1388 5423 3ee7 fa86 cd05  ........T#>.....
        0x0020:  8018 0140 fe2f 0000 0101 080a c866 c3dc  ...@./.......f..
        0x0030:  2d60 bcee 5f74 7261 6465 72              -`.._trader
04:15:06.520761 IP 127.0.0.1.5000 > 127.0.0.1.49900: Flags [.], ack 24, win 320, options [nop,
nop,TS val 761314743 ecr 3362178012], length 0
        0x0000:  4500 0034 0000 4000 4006 0000 7f00 0001  E..4..@.@.......
        0x0010:  7f00 0001 1388 c2ec fa86 cd05 5423 3eee  ............T#>.
        0x0020:  8010 0140 fe28 0000 0101 080a 2d60 bdb7  ...@.(......-`..
        0x0030:  c866 c3dc                                .f..
04:15:06.688994 IP 127.0.0.1.49900 > 127.0.0.1.5000: Flags [P.], seq 24:25, ack 1, win 320, op
tions [nop,nop,TS val 3362178212 ecr 761314743], length 1
        0x0000:  4500 0035 0000 4000 4006 0000 7f00 0001  E..5..@.@.......
        0x0010:  7f00 0001 c2ec 1388 5423 3eee fa86 cd05  ........T#>.....
        0x0020:  8018 0140 fe29 0000 0101 080a c866 c4a4  ...@.).......f..
        0x0030:  2d60 bdb7 0a                             -`...
04:15:06.690381 IP 127.0.0.1.5000 > 127.0.0.1.49900: Flags [P.], seq 1:4, ack 25, win 320, opt
ions [nop,nop,TS val 761314913 ecr 3362178212], length 3
        0x0000:  4500 0037 0000 4000 4006 0000 7f00 0001  E..7..@.@.......
        0x0010:  7f00 0001 1388 c2ec fa86 cd05 5423 3eef  ............T#>.
        0x0020:  8018 0140 fe2b 0000 0101 080a 2d60 be61  ...@.+......-`.a
        0x0030:  c866 c4a4 4f4b 0a                        .f..OK.
04:15:06.737661 IP 127.0.0.1.49900 > 127.0.0.1.5000: Flags [.], ack 4, win 320, options [nop,n
op,TS val 3362178265 ecr 761314913], length 0
        0x0000:  4500 0034 0000 4000 4006 0000 7f00 0001  E..4..@.@.......
        0x0010:  7f00 0001 c2ec 1388 5423 3eef fa86 cd08  ........T#>.....
        0x0020:  8010 0140 fe28 0000 0101 080a c866 c4d9  ...@.(.......f..
        0x0030:  2d60 be61                                -`.a
04:15:17.197358 IP 127.0.0.1.49900 > 127.0.0.1.5000: Flags [F.], seq 25, ack 4, win 320, optio
ns [nop,nop,TS val 3362188725 ecr 761314913], length 0
```

Figure 5: `tcpdump -X` — the four data segments (6, 10, 7, 1 bytes) spelling `LOGIN/experiment/_trader/\n`, then the server's `OK\n` reply

```
04:15:06.520761 IP 127.0.0.1.5000 > 127.0.0.1.49900: Flags [.], ack 24, win 320, options [nop,
nop,TS val 761314743 ecr 3362178012], length 0
        0x0000:  4500 0034 0000 4000 4006 0000 7f00 0001  E..4..@.@.......
        0x0010:  7f00 0001 1388 c2ec fa86 cd05 5423 3eee  ............T#>.
        0x0020:  8010 0140 fe28 0000 0101 080a 2d60 bdb7  ...@.(......-`..
        0x0030:  c866 c3dc                                .f..
04:15:06.688994 IP 127.0.0.1.49900 > 127.0.0.1.5000: Flags [P.], seq 24:25, ack 1, win 320, op
tions [nop,nop,TS val 3362178212 ecr 761314743], length 1
        0x0000:  4500 0035 0000 4000 4006 0000 7f00 0001  E..5..@.@.......
        0x0010:  7f00 0001 c2ec 1388 5423 3eee fa86 cd05  ........T#>.....
        0x0020:  8018 0140 fe29 0000 0101 080a c866 c4a4  ...@.).......f..
        0x0030:  2d60 bdb7 0a                             -`...
04:15:06.690381 IP 127.0.0.1.5000 > 127.0.0.1.49900: Flags [P.], seq 1:4, ack 25, win 320, opt
ions [nop,nop,TS val 761314913 ecr 3362178212], length 3
        0x0000:  4500 0037 0000 4000 4006 0000 7f00 0001  E..7..@.@.......
        0x0010:  7f00 0001 1388 c2ec fa86 cd05 5423 3eef  ............T#>.
        0x0020:  8018 0140 fe2b 0000 0101 080a 2d60 be61  ...@.+......-`.a
        0x0030:  c866 c4a4 4f4b 0a                        .f..OK.
04:15:06.737661 IP 127.0.0.1.49900 > 127.0.0.1.5000: Flags [.], ack 4, win 320, options [nop,n
op,TS val 3362178265 ecr 761314913], length 0
        0x0000:  4500 0034 0000 4000 4006 0000 7f00 0001  E..4..@.@.......
        0x0010:  7f00 0001 c2ec 1388 5423 3eef fa86 cd08  ........T#>.....
        0x0020:  8010 0140 fe28 0000 0101 080a c866 c4d9  ...@.(.......f..
        0x0030:  2d60 be61                                -`.a
04:15:17.197358 IP 127.0.0.1.49900 > 127.0.0.1.5000: Flags [F.], seq 25, ack 4, win 320, optio
ns [nop,nop,TS val 3362188725 ecr 761314913], length 0
        0x0000:  4500 0034 0000 4000 4006 0000 7f00 0001  E..4..@.@.......
        0x0010:  7f00 0001 c2ec 1388 5423 3eef fa86 cd08  ........T#>.....
        0x0020:  8011 0140 fe28 0000 0101 080a c866 edb5  ...@.(.......f..
        0x0030:  2d60 be61                                -`.a
04:15:17.197384 IP 127.0.0.1.5000 > 127.0.0.1.49900: Flags [.], ack 26, win 320, options [nop,
nop,TS val 761325420 ecr 3362188725], length 0
        0x0000:  4500 0034 0000 4000 4006 0000 7f00 0001  E..4..@.@.......
        0x0010:  7f00 0001 1388 c2ec fa86 cd08 5423 3ef0  ............T#>.
        0x0020:  8010 0140 fe28 0000 0101 080a 2d60 e76c  ...@.(......-`.l
        0x0030:  c866 edb5                                .f..
04:15:17.197423 IP 127.0.0.1.5000 > 127.0.0.1.49900: Flags [F.], seq 4, ack 26, win 320, optio
ns [nop,nop,TS val 761325420 ecr 3362188725], length 0
        0x0000:  4500 0034 0000 4000 4006 0000 7f00 0001  E..4..@.@.......
        0x0010:  7f00 0001 1388 c2ec fa86 cd08 5423 3ef0  ............T#>.
        0x0020:  8011 0140 fe28 0000 0101 080a 2d60 e76c  ...@.(......-`.l
        0x0030:  c866 edb5                                .f..
04:15:17.197428 IP 127.0.0.1.49900 > 127.0.0.1.5000: Flags [.], ack 5, win 320, options [nop,n
op,TS val 3362188725 ecr 761325420], length 0
        0x0000:  4500 0034 0000 4000 4006 0000 7f00 0001  E..4..@.@.......
        0x0010:  7f00 0001 c2ec 1388 5423 3ef0 fa86 cd09  ........T#>.....
        0x0020:  8010 0140 fe28 0000 0101 080a c866 edb5  ...@.(.......f..
        0x0030:  2d60 e76c                                -`.l
^C
22 packets captured
22 packets received by filter
0 packets dropped by kernel
```

Figure 6: `tcpdump -X` — connection close; `22 packets captured, 0 dropped by kernel`

Answer. The message arrived in pieces, not as one unit. Each segment carried only the bytes written so far. The server appended them to that connection's input buffer and did nothing until the `\n` appeared in the buffer; then it parsed `LOGIN experiment_trader` and replied `OK\n`. This shows that TCP is an ordered byte stream with no message boundaries: one `send()` on the client does not become one `recv()` on the server, and a message can be split anywhere. The application must add its own framing, which here is the newline delimiter. (TCP is also free to *combine* writes into one segment; this run happened to keep them separate because of the 0.2 s

gaps.)

Experiment 4 — One Client Should Not Stall the Others

Run: `python3 experiment.py 4`

Question. When Client 1 remains connected but sends no data, can the Exchange Server still accept and service Client 2? Identify the server operation that determines the answer.

Approach. Client 1 connects and sends a partial line (`LOGIN blocked_client`, no newline), then stays silent. Two seconds later Client 2 connects and sends a complete `LOGIN active_client\n`; the harness times how long the `OK` takes. While Client 1 is idle we list the server's open sockets with `procstat` to confirm both connections exist.

Commands / tools.

```
procstat -f $(pgrep exchange_server)        # the server's open sockets
sockstat -4 | grep ':5000'
```

Observation. The harness reported `Client 2 response: 'OK'` with `Elapsed time: 0.000 seconds`, even though Client 1 had sent only `LOGIN blocked_client` (no newline) and then gone silent.

`procstat -f` on the server (PID 6712) showed three TCP sockets at that moment:

```
FD 4  TCP  127.0.0.1:5000   *:0                 (listening)
FD 5  TCP  127.0.0.1:5000   127.0.0.1:37591     (Client 1)
FD 6  TCP  127.0.0.1:5000   127.0.0.1:53905     (Client 2)
```

So both clients were connected at the same time, and Client 2 got its reply immediately. (`truss` attached but did not print useful syscall lines before we detached, so we do not rely on it here.)

```
[root@COL334 Assignment:~/COL334-A2 # python3 experiment.py 4
=== Experiment 4: One Client Should Not Stall the Others ===

Started Exchange Server (PID 6712).
fd limit (soft): 200000
Exchange Server listening on 127.0.0.1:5000 (event loop: poll)
Exchange Server is listening.

Client 1: connected from 127.0.0.1:37591

Client 1 will send an incomplete application message.
Client 1 will now remain silent.

Client 2: connected from 127.0.0.1:53905
Client 2 will send a complete application message.

Client 2 response: 'OK'
Elapsed time: 0.000 seconds

Investigate the behavior of both TCP connections.
Press Ctrl-C when you are finished.
^C
Experiment finished.
Stopping Exchange Server...
```

Figure 7: harness output — `Client 2 response: 'OK'`, `Elapsed time: 0.000 seconds`; Client 1 on :37591, Client 2 on :53905

```
[root@COL334 Assignment:~/COL334-A2 # truss -p 6712                                  ]
[^Croot@COL334 Assignment:~/COL334-A2 # ps -p 6712 -o pid,stat,command               ]
  PID STAT COMMAND
 6712   Ss /root/COL334-A2/exchange_server 127.0.0.1 5000
[root@COL334 Assignment:~/COL334-A2 # procstat -f 6712                               ]
   PID COMM                FD T V FLAGS    REF  OFFSET PRO NAME
  6712 exchange_server   text v r r-------   -       - -   /root/COL334-A2/exchange_server
  6712 exchange_server    cwd v d r-------   -       - -   /root/COL334-A2
  6712 exchange_server   root v d r-------   -       - -   /
  6712 exchange_server      0 v c rw------   9   22115 -   /dev/pts/0
  6712 exchange_server      1 v c rw------   9   22115 -   /dev/pts/0
  6712 exchange_server      2 v c rw------   9   22115 -   /dev/pts/0
  6712 exchange_server      3 v c r-------   1       0 -   /dev/null
  6712 exchange_server      4 s - rw---n--   1       0 TCP 0 0 127.0.0.1:5000 *:0
  6712 exchange_server      5 s - rw---n--   1       0 TCP 0 0 127.0.0.1:5000 127.0.0.1:37591
  6712 exchange_server      6 s - rw---n--   1       0 TCP 0 0 127.0.0.1:5000 127.0.0.1:53905
```

Figure 8: `procstat -f` for the server (PID 6712) — FD 4 listening, FD 5 → :37591 (idle Client 1), FD 6 → :53905 (Client 2)

Answer. Yes. Client 2 was served in 0.000 s while Client 1 sat idle with an incomplete message. The operation that decides this is the server's `poll()` call. It is the only place the server blocks, and it waits on every socket at once. All sockets are non-blocking, so the server never sits inside `recv()` or `send()` for one client. Client 1 having no data just means `poll()` does not list it as ready; `poll()` still lists the listening socket and Client 2, which are handled normally. A blocking `recv()` on Client 1 would have frozen the whole server. The server also prints `event loop: poll` at startup, and the source (`src/server.cpp`, `poll_once()`) confirms the mechanism.

Experiment 5 — Multiple Clients and I/O Multiplexing *(optional)*

Run: `python3 experiment.py 5`

Question. At a particular point during the experiment, which client connections are actually ready for the server to service, and what evidence from the running system allows you to determine this?

Approach. Five connections are opened (client ports 14226, 26960, 27683, 30524, 57599); clients 1, 3, and 5 send a `LOGIN`, clients 2 and 4 stay idle. We list the server's sockets, look at the per-socket queue lengths in `netstat`, and watch the server with `truss`.

Commands / tools.

```
sockstat -4 -p 5000
netstat -an -p tcp | grep '\.5000'          # Recv-Q / Send-Q columns
truss -p $(pgrep exchange_server)
```

Observation. `sockstat` showed the server (PID 6769) holding six sockets: FD 4 listening, and FD 5–9 for the five clients. All five connections were `ESTABLISHED` in `netstat`. On the server side every `Recv-Q` was 0 — the server had already read and replied to clients 1, 3, and 5. On the client side, the three that sent a `LOGIN` (ports 14226, 27683, 57599) each showed `Recv-Q = 3`, the unread `OK\n` reply; the two idle clients (26960, 30524) showed `Recv-Q = 0`. `truss` showed the server making a single `recvfrom` on one client fd and otherwise sitting idle until the experiment stopped it.

```
root@COL334 Assignment:~/COL334-A2 # sockstat -4 -p 5000
USER COMMAND     PID FD PROTO LOCAL ADDRESS         FOREIGN ADDRESS
root exchange_s 6769  4 tcp4  127.0.0.1:5000        *:*
root exchange_s 6769  5 tcp4  127.0.0.1:5000        127.0.0.1:14226
root exchange_s 6769  6 tcp4  127.0.0.1:5000        127.0.0.1:26960
root exchange_s 6769  7 tcp4  127.0.0.1:5000        127.0.0.1:27683
root exchange_s 6769  8 tcp4  127.0.0.1:5000        127.0.0.1:30524
root exchange_s 6769  9 tcp4  127.0.0.1:5000        127.0.0.1:57599
root python3.12 6768  3 tcp4  127.0.0.1:14226       127.0.0.1:5000
root python3.12 6768  4 tcp4  127.0.0.1:26960       127.0.0.1:5000
root python3.12 6768  5 tcp4  127.0.0.1:27683       127.0.0.1:5000
root python3.12 6768  6 tcp4  127.0.0.1:30524       127.0.0.1:5000
root python3.12 6768  7 tcp4  127.0.0.1:57599       127.0.0.1:5000
root@COL334 Assignment:~/COL334-A2 # netstat -an -p tcp | grep 5000
tcp4           0      0 127.0.0.1.5000         127.0.0.1.57599        ESTABLISHED
tcp4           3      0 127.0.0.1.57599        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         127.0.0.1.30524        ESTABLISHED
tcp4           0      0 127.0.0.1.30524        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         127.0.0.1.27683        ESTABLISHED
tcp4           3      0 127.0.0.1.27683        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         127.0.0.1.26960        ESTABLISHED
tcp4           0      0 127.0.0.1.26960        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         127.0.0.1.14226        ESTABLISHED
tcp4           3      0 127.0.0.1.14226        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         *.*                    LISTEN
root@COL334 Assignment:~/COL334-A2 # truss -p 6769
recvfrom(5,0x81077780,16384,0,NULL,0x0)          = 0 (0x0)
SIGNAL 15 (SIGTERM) code=SI_USER pid=6768 uid=0
process killed, signal = 15
```

Figure 9: `sockstat` (server PID 6769, FD 4 listen + FD 5–9 for the five clients) and `netstat` — five `ESTABLISHED`, `Recv-Q = 3` only on :14226, :27683, :57599 (the clients that sent a `LOGIN`); the `truss` line shows `recvfrom(...) = 0` then `SIGTERM`

Answer. All five connections are `ESTABLISHED`, but only the ones with data to move are ever "ready" for the server. Clients 1, 3, and 5 sent a `LOGIN`; the server was woken for those sockets, read the request, replied `OK`, and went back to waiting. Clients 2 and 4 sent nothing, so `poll()` never reports them and the server does no work for them. At the moment of capture the server-side `Recv-Q` is 0 everywhere (all pending input already consumed); the only trace of activity is the 3-byte `OK` reply queued in the receive buffers of the three clients that spoke. This shows that "connected" and "ready for service" are not the same thing — the server only touches a socket when that socket has something to read or write.

Experiment 6 — FIN vs. RST: Orderly and Abrupt Termination

Run: `python3 experiment.py 6`

Question. How does an abrupt client termination differ from the orderly shutdown? Identify the TCP event observed on the network and the corresponding behaviour of the Exchange Server's socket.

Approach. Part A: the client calls `shutdown(SHUT_WR)` (orderly). Part B: a second client sets `SO_LINGER` to `{on, 0}` and closes (abortive). Capture the packets and the server-side socket state for each.

Commands / tools.

```
tcpdump -i lo0 -n -S 'tcp port 5000'        # compare F (FIN) vs R (RST)
netstat -an -p tcp | grep '\.5000'          # socket state after each part
```

Observation. The orderly client used port 19512, the abrupt client port 50393.

Part A (orderly). `tcpdump` showed the client send a **FIN** (`Flags [F.]`) at `04:30:27.961625`; the server acknowledged it, sent its own **FIN** immediately after (`.961639`), and the client acknowledged that — a normal four-way close. `netstat` right after showed the connection briefly as `CLOSED`, then gone:

```
127.0.0.1.19512   127.0.0.1.5000    CLOSED
127.0.0.1.5000    *.*               LISTEN
```

Part B (abrupt). `tcpdump` showed the client send a single **RST** (`Flags [R.]`) at `04:30:37.965760` — no FIN, no acknowledgement, no four-way exchange. `netstat` right after showed nothing left but the listening socket:

```
127.0.0.1.5000    *.*               LISTEN
```

17 packets captured, 0 dropped by the kernel.

```
root@COL334 Assignment:~/COL334-A2 # tcpdump -i lo0 -n -S port 5000
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on lo0, link-type NULL (BSD loopback), snapshot length 262144 bytes
04:30:27.857807 IP 127.0.0.1.31568 > 127.0.0.1.5000: Flags [S], seq 3377931253, win 65535, opt
ions [mss 16344,nop,wscale 8,sackOK,TS val 3692486540 ecr 0], length 0
04:30:27.857853 IP 127.0.0.1.5000 > 127.0.0.1.31568: Flags [R.], seq 0, ack 3377931254, win 0,
 length 0
04:30:27.961055 IP 127.0.0.1.28248 > 127.0.0.1.5000: Flags [S], seq 2093079559, win 65535, opt
ions [mss 16344,nop,wscale 8,sackOK,TS val 2147623994 ecr 0], length 0
04:30:27.961154 IP 127.0.0.1.5000 > 127.0.0.1.28248: Flags [S.], seq 2626412642, ack 209307956
0, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 1791679297 ecr 2147623994], length
 0
04:30:27.961181 IP 127.0.0.1.28248 > 127.0.0.1.5000: Flags [.], ack 2626412643, win 320, optio
ns [nop,nop,TS val 2147623994 ecr 1791679297], length 0
04:30:27.961281 IP 127.0.0.1.28248 > 127.0.0.1.5000: Flags [R.], seq 2093079560, ack 262641264
3, win 0, options [nop,nop,TS val 2147623994 ecr 1791679297], length 0
04:30:27.961435 IP 127.0.0.1.19512 > 127.0.0.1.5000: Flags [S], seq 358365818, win 65535, opti
ons [mss 16344,nop,wscale 8,sackOK,TS val 312345549 ecr 0], length 0
04:30:27.961512 IP 127.0.0.1.5000 > 127.0.0.1.19512: Flags [S.], seq 2728944220, ack 358365819
, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 3014746165 ecr 312345549], length 0
04:30:27.961559 IP 127.0.0.1.19512 > 127.0.0.1.5000: Flags [.], ack 2728944221, win 320, optio
ns [nop,nop,TS val 312345549 ecr 3014746165], length 0
04:30:27.961625 IP 127.0.0.1.19512 > 127.0.0.1.5000: Flags [F.], seq 358365819, ack 2728944221
, win 320, options [nop,nop,TS val 312345549 ecr 3014746165], length 0
04:30:27.961630 IP 127.0.0.1.5000 > 127.0.0.1.19512: Flags [.], ack 358365820, win 320, option
s [nop,nop,TS val 3014746165 ecr 312345549], length 0
04:30:27.961639 IP 127.0.0.1.5000 > 127.0.0.1.19512: Flags [F.], seq 2728944221, ack 358365820
, win 320, options [nop,nop,TS val 3014746165 ecr 312345549], length 0
04:30:27.961643 IP 127.0.0.1.19512 > 127.0.0.1.5000: Flags [.], ack 2728944222, win 320, optio
ns [nop,nop,TS val 312345549 ecr 3014746165], length 0
04:30:37.965156 IP 127.0.0.1.50393 > 127.0.0.1.5000: Flags [S], seq 1753329897, win 65535, opt
ions [mss 16344,nop,wscale 8,sackOK,TS val 558150407 ecr 0], length 0
04:30:37.965298 IP 127.0.0.1.5000 > 127.0.0.1.50393: Flags [S.], seq 830534095, ack 1753329898
, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 2859588565 ecr 558150407], length 0
04:30:37.965337 IP 127.0.0.1.50393 > 127.0.0.1.5000: Flags [.], ack 830534096, win 320, option
s [nop,nop,TS val 558150407 ecr 2859588565], length 0
04:30:37.965760 IP 127.0.0.1.50393 > 127.0.0.1.5000: Flags [R.], seq 1753329898, ack 830534096
, win 0, options [nop,nop,TS val 558150407 ecr 2859588565], length 0
^C
17 packets captured
17 packets received by filter
0 packets dropped by kernel
```

Figure 10: `tcpdump -S` — Part A: `[F.]` from :19512 at 04:30:27.961625, server `[F.]` back at .961639, then ACK (four-way close). Part B: a lone `[R.]` from :50393 at 04:30:37.965760. 17 packets captured, 0 dropped

```
[root@COL334 Assignment:~/COL334-A2 # netstat -an -p tcp | grep 5000
 you have mail
 tcp4           0      0 127.0.0.1.19512        127.0.0.1.5000         CLOSED
 tcp4           0      0 127.0.0.1.5000         *.*                    LISTEN
[root@COL334 Assignment:~/COL334-A2 # netstat -an -p tcp | grep 5000
 tcp4           0      0 127.0.0.1.5000         *.*                    LISTEN
```

Figure 11: `netstat` — Part A leaves a transient `127.0.0.1.19512 … CLOSED` row, then only `LISTEN` remains after Part B

Answer. * **Orderly (FIN).** The client sends a FIN. The server's `recv()` returns 0, the server closes its end, and both sides exchange FIN/ACK. The connection drains cleanly and then leaves the socket table (we caught it as `CLOSED` just before it disappeared). No data is lost. * **Abrupt (RST).** The client sends a single RST. There is no FIN and no four-way close. The server learns of it through `poll()` (`POLLHUP/POLLERR`) or a failed `recv()/send()` (`ECONNRESET/EPIPE`), marks the connection closing, and calls `close_client()` to drop it from the poller and the order and subscriber tables. The socket vanishes from `netstat` immediately, and any bytes still in its receive buffer are discarded. Per §2.6 a resting order from that client stays in the book.

Experiment 7 — Backpressure and the Slow Receiver

Run: `python3 experiment.py 7`

Question. How does the Exchange Server's TCP connection to the slow client behave as the client stops reading, and what evidence shows whether this eventually affects the server's ability to communicate with other clients?

Approach. The harness runs two traders and two Market-Data clients subscribed to `JNST`; one MD client keeps reading, the other never reads its socket. Trades are generated continuously. We sampled `netstat -an -p tcp` once a second with a shell loop and also captured the traffic with `tcpdump`.

Commands / tools.

```
while true; do date; netstat -an -p tcp | grep 5000; echo ---; sleep 1; done
tcpdump -i lo0 -n -S 'tcp port 5000'
```

Observation. The connections were: traders on ports 44253 and 43559, the normal Market-Data client on 61768, and the **slow** Market-Data client on **61256**. Over a 14-second window the slow client's **receive queue** (`netstat` row `127.0.0.1.61256` → `127.0.0.1.5000`, `Recv-Q`) grew steadily:

04:35:07	850	04:35:12	3247	04:35:17	5593
04:35:08	1343	04:35:13	3723	04:35:18	6069
04:35:09	1836	04:35:14	4216	04:35:19	6528
04:35:10	2312	04:35:15	4675	04:35:20	7004
04:35:11	2788	04:35:16	5134		

That is about +475 bytes/second — the `TRADE` messages the server keeps sending, piling up unread in the slow client's kernel receive buffer. Over the same window:

- the server's **send queue** toward the slow client (`127.0.0.1.5000` → `127.0.0.1.61256`, `Send-Q`) stayed small — mostly 17, sometimes 0 — it did **not** grow;
- the normal Market-Data client (61768) showed `Recv-Q = 0` and `Send-Q = 0` the entire time;
- the traders (44253, 43559) kept exchanging orders normally.

`tcpdump` showed the server continuously pushing `TRADE` segments to every client and the normal client acknowledging them with an advancing sequence number throughout.

```
root@COL334 Assignment:~/COL334-A2 # while true; do
>     date
>     netstat -an -p tcp | grep 5000
>     echo "----------------------------------------"
>     sleep 1
> done
Tue Sep  8 04:35:07 IST 2026
tcp4           0     34 127.0.0.1.5000         127.0.0.1.44253        ESTABLISHED
tcp4           0      0 127.0.0.1.44253        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         127.0.0.1.61768        ESTABLISHED
tcp4           0      0 127.0.0.1.61768        127.0.0.1.5000         ESTABLISHED
tcp4           0     17 127.0.0.1.5000         127.0.0.1.61256        ESTABLISHED
tcp4         850      0 127.0.0.1.61256        127.0.0.1.5000         ESTABLISHED
tcp4           0     17 127.0.0.1.5000         127.0.0.1.43559        ESTABLISHED
tcp4           0      0 127.0.0.1.43559        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         *.*                    LISTEN
----------------------------------------
Tue Sep  8 04:35:08 IST 2026
tcp4           0     35 127.0.0.1.5000         127.0.0.1.44253        ESTABLISHED
tcp4           0      0 127.0.0.1.44253        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         127.0.0.1.61768        ESTABLISHED
tcp4           0      0 127.0.0.1.61768        127.0.0.1.5000         ESTABLISHED
tcp4           0     17 127.0.0.1.5000         127.0.0.1.61256        ESTABLISHED
tcp4        1343      0 127.0.0.1.61256        127.0.0.1.5000         ESTABLISHED
tcp4           0     17 127.0.0.1.5000         127.0.0.1.43559        ESTABLISHED
tcp4           0      0 127.0.0.1.43559        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         *.*                    LISTEN
----------------------------------------
Tue Sep  8 04:35:09 IST 2026
tcp4           0     35 127.0.0.1.5000         127.0.0.1.44253        ESTABLISHED
tcp4          35      0 127.0.0.1.44253        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         127.0.0.1.61768        ESTABLISHED
tcp4           0      0 127.0.0.1.61768        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         127.0.0.1.61256        ESTABLISHED
tcp4        1836      0 127.0.0.1.61256        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         127.0.0.1.43559        ESTABLISHED
tcp4          17      0 127.0.0.1.43559        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         *.*                    LISTEN
----------------------------------------
Tue Sep  8 04:35:10 IST 2026
tcp4           0     35 127.0.0.1.5000         127.0.0.1.44253        ESTABLISHED
tcp4           0      0 127.0.0.1.44253        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         127.0.0.1.61768        ESTABLISHED
tcp4           0      0 127.0.0.1.61768        127.0.0.1.5000         ESTABLISHED
tcp4           0     17 127.0.0.1.5000         127.0.0.1.61256        ESTABLISHED
tcp4        2312      0 127.0.0.1.61256        127.0.0.1.5000         ESTABLISHED
tcp4           0     17 127.0.0.1.5000         127.0.0.1.43559        ESTABLISHED
tcp4           0      0 127.0.0.1.43559        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         *.*                    LISTEN
----------------------------------------
Tue Sep  8 04:35:11 IST 2026
tcp4           0     35 127.0.0.1.5000         127.0.0.1.44253        ESTABLISHED
```

Figure 12: `netstat` loop 04:35:07–11 — slow client :61256 `Recv-Q` climbing 850 → 2312 while every other row stays at 0

```
----------------------------------------
Tue Sep  8 04:35:11 IST 2026
tcp4           0     35 127.0.0.1.5000         127.0.0.1.44253        ESTABLISHED
tcp4          35      0 127.0.0.1.44253        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         127.0.0.1.61768        ESTABLISHED
tcp4           0      0 127.0.0.1.61768        127.0.0.1.5000         ESTABLISHED
tcp4           0     17 127.0.0.1.5000         127.0.0.1.61256        ESTABLISHED
tcp4        2788      0 127.0.0.1.61256        127.0.0.1.5000         ESTABLISHED
tcp4           0     17 127.0.0.1.5000         127.0.0.1.43559        ESTABLISHED
tcp4          17      0 127.0.0.1.43559        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         *.*                    LISTEN
----------------------------------------
Tue Sep  8 04:35:12 IST 2026
tcp4           0     35 127.0.0.1.5000         127.0.0.1.44253        ESTABLISHED
tcp4           0      0 127.0.0.1.44253        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         127.0.0.1.61768        ESTABLISHED
tcp4           0      0 127.0.0.1.61768        127.0.0.1.5000         ESTABLISHED
tcp4           0     17 127.0.0.1.5000         127.0.0.1.61256        ESTABLISHED
tcp4        3247      0 127.0.0.1.61256        127.0.0.1.5000         ESTABLISHED
tcp4           0     17 127.0.0.1.5000         127.0.0.1.43559        ESTABLISHED
tcp4           0      0 127.0.0.1.43559        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         *.*                    LISTEN
----------------------------------------
Tue Sep  8 04:35:13 IST 2026
tcp4           0     35 127.0.0.1.5000         127.0.0.1.44253        ESTABLISHED
tcp4           0      0 127.0.0.1.44253        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         127.0.0.1.61768        ESTABLISHED
tcp4           0      0 127.0.0.1.61768        127.0.0.1.5000         ESTABLISHED
tcp4           0     17 127.0.0.1.5000         127.0.0.1.61256        ESTABLISHED
tcp4        3723      0 127.0.0.1.61256        127.0.0.1.5000         ESTABLISHED
tcp4           0     17 127.0.0.1.5000         127.0.0.1.43559        ESTABLISHED
tcp4           0      0 127.0.0.1.43559        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         *.*                    LISTEN
----------------------------------------
Tue Sep  8 04:35:14 IST 2026
tcp4           0     35 127.0.0.1.5000         127.0.0.1.44253        ESTABLISHED
tcp4          35      0 127.0.0.1.44253        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         127.0.0.1.61768        ESTABLISHED
tcp4           0      0 127.0.0.1.61768        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         127.0.0.1.61256        ESTABLISHED
tcp4        4216      0 127.0.0.1.61256        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         127.0.0.1.43559        ESTABLISHED
tcp4          17      0 127.0.0.1.43559        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         *.*                    LISTEN
----------------------------------------
Tue Sep  8 04:35:15 IST 2026
tcp4           0     35 127.0.0.1.5000         127.0.0.1.44253        ESTABLISHED
tcp4          35      0 127.0.0.1.44253        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         127.0.0.1.61768        ESTABLISHED
tcp4           0      0 127.0.0.1.61768        127.0.0.1.5000         ESTABLISHED
tcp4           0     17 127.0.0.1.5000         127.0.0.1.61256        ESTABLISHED
tcp4        4675      0 127.0.0.1.61256        127.0.0.1.5000         ESTABLISHED
tcp4           0     17 127.0.0.1.5000         127.0.0.1.43559        ESTABLISHED
tcp4          17      0 127.0.0.1.43559        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         *.*                    LISTEN
----------------------------------------
Tue Sep  8 04:35:16 IST 2026
```

Figure 13: `netstat` loop 04:35:11–16 — :61256 `Recv-Q` 2788 → 4675, others still 0

```
tcp4           0     35 127.0.0.1.5000         127.0.0.1.44253        ESTABLISHED
tcp4           0      0 127.0.0.1.44253        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         127.0.0.1.61768        ESTABLISHED
tcp4           0      0 127.0.0.1.61768        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         127.0.0.1.61256        ESTABLISHED
tcp4        5134      0 127.0.0.1.61256        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         127.0.0.1.43559        ESTABLISHED
tcp4           0      0 127.0.0.1.43559        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         *.*                    LISTEN
--------------------------------------
Tue Sep  8 04:35:17 IST 2026
tcp4           0     35 127.0.0.1.5000         127.0.0.1.44253        ESTABLISHED
tcp4           0      0 127.0.0.1.44253        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         127.0.0.1.61768        ESTABLISHED
tcp4           0      0 127.0.0.1.61768        127.0.0.1.5000         ESTABLISHED
tcp4           0     17 127.0.0.1.5000         127.0.0.1.61256        ESTABLISHED
tcp4        5593      0 127.0.0.1.61256        127.0.0.1.5000         ESTABLISHED
tcp4           0     17 127.0.0.1.5000         127.0.0.1.43559        ESTABLISHED
tcp4           0      0 127.0.0.1.43559        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         *.*                    LISTEN
--------------------------------------
Tue Sep  8 04:35:18 IST 2026
tcp4           0     35 127.0.0.1.5000         127.0.0.1.44253        ESTABLISHED
tcp4          35      0 127.0.0.1.44253        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         127.0.0.1.61768        ESTABLISHED
tcp4           0      0 127.0.0.1.61768        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         127.0.0.1.61256        ESTABLISHED
tcp4        6069      0 127.0.0.1.61256        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         127.0.0.1.43559        ESTABLISHED
tcp4          17      0 127.0.0.1.43559        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         *.*                    LISTEN
--------------------------------------
Tue Sep  8 04:35:19 IST 2026
tcp4           0     35 127.0.0.1.5000         127.0.0.1.44253        ESTABLISHED
tcp4           0      0 127.0.0.1.44253        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         127.0.0.1.61768        ESTABLISHED
tcp4           0      0 127.0.0.1.61768        127.0.0.1.5000         ESTABLISHED
tcp4           0     17 127.0.0.1.5000         127.0.0.1.61256        ESTABLISHED
tcp4        6528      0 127.0.0.1.61256        127.0.0.1.5000         ESTABLISHED
tcp4           0     17 127.0.0.1.5000         127.0.0.1.43559        ESTABLISHED
tcp4           0      0 127.0.0.1.43559        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         *.*                    LISTEN
--------------------------------------
Tue Sep  8 04:35:20 IST 2026
tcp4           0     35 127.0.0.1.5000         127.0.0.1.44253        ESTABLISHED
tcp4           0      0 127.0.0.1.44253        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         127.0.0.1.61768        ESTABLISHED
tcp4           0      0 127.0.0.1.61768        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         127.0.0.1.61256        ESTABLISHED
tcp4        7004      0 127.0.0.1.61256        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         127.0.0.1.43559        ESTABLISHED
tcp4           0      0 127.0.0.1.43559        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         *.*                    LISTEN
--------------------------------------
Tue Sep  8 04:35:21 IST 2026
tcp4           0     35 127.0.0.1.5000         127.0.0.1.44253        ESTABLISHED
tcp4           0      0 127.0.0.1.44253        127.0.0.1.5000         ESTABLISHED
tcp4           0      0 127.0.0.1.5000         127.0.0.1.61768        ESTABLISHED
```

Figure 14: `netstat` loop 04:35:16–21 — :61256 `Recv-Q` 5134 → 7004; the normal MD client and both traders remain at `Recv-Q = Send-Q = 0`

```
04:42:56.795255 IP 127.0.0.1.25024 > 127.0.0.1.5000: Flags [.], ack 1, win 320, options [nop,n
op,TS val 972530752 ecr 3163228115], length 0
04:42:56.795291 IP 127.0.0.1.25024 > 127.0.0.1.5000: Flags [P.], seq 1:16, ack 1, win 320, opt
ions [nop,nop,TS val 972530752 ecr 3163228115], length 15
04:42:56.795330 IP 127.0.0.1.5000 > 127.0.0.1.25024: Flags [P.], seq 1:4, ack 16, win 320, opt
ions [nop,nop,TS val 3163228115 ecr 972530752], length 3
04:42:56.830687 IP 127.0.0.1.56877 > 127.0.0.1.5000: Flags [.], ack 4, win 320, options [nop,n
op,TS val 1567612329 ecr 1979854590], length 0
04:42:56.830700 IP 127.0.0.1.14353 > 127.0.0.1.5000: Flags [.], ack 4, win 320, options [nop,n
op,TS val 1015180009 ecr 105271306], length 0
04:42:56.830702 IP 127.0.0.1.25024 > 127.0.0.1.5000: Flags [.], ack 4, win 320, options [nop,n
op,TS val 972530793 ecr 3163228115], length 0
04:42:57.775946 IP 127.0.0.1.14353 > 127.0.0.1.5000: Flags [P.], seq 24:39, ack 4, win 320, op
tions [nop,nop,TS val 1015180952 ecr 105271306], length 15
04:42:57.776004 IP 127.0.0.1.56877 > 127.0.0.1.5000: Flags [P.], seq 25:41, ack 4, win 320, op
tions [nop,nop,TS val 1567613272 ecr 1979854590], length 16
04:42:57.776117 IP 127.0.0.1.5000 > 127.0.0.1.14353: Flags [P.], seq 4:21, ack 39, win 320, op
tions [nop,nop,TS val 105272309 ecr 1015180952], length 17
04:42:57.776144 IP 127.0.0.1.5000 > 127.0.0.1.56877: Flags [P.], seq 4:37, ack 41, win 320, op
tions [nop,nop,TS val 1979855593 ecr 1567613272], length 33
04:42:57.776152 IP 127.0.0.1.5000 > 127.0.0.1.49609: Flags [P.], seq 4:21, ack 16, win 320, op
tions [nop,nop,TS val 66984834 ecr 2066070043], length 17
04:42:57.776155 IP 127.0.0.1.5000 > 127.0.0.1.14353: Flags [P.], seq 21:39, ack 39, win 320, o
ptions [nop,nop,TS val 105272309 ecr 1015180952], length 18
04:42:57.776158 IP 127.0.0.1.5000 > 127.0.0.1.25024: Flags [P.], seq 4:21, ack 16, win 320, op
tions [nop,nop,TS val 3163229099 ecr 972530793], length 17
04:42:57.776185 IP 127.0.0.1.14353 > 127.0.0.1.5000: Flags [.], ack 39, win 320, options [nop,
nop,TS val 1015180952 ecr 105272309], length 0
04:42:57.834912 IP 127.0.0.1.25024 > 127.0.0.1.5000: Flags [.], ack 21, win 320, options [nop,
nop,TS val 972531797 ecr 3163229099], length 0
04:42:57.834936 IP 127.0.0.1.49609 > 127.0.0.1.5000: Flags [.], ack 21, win 320, options [nop,
nop,TS val 2066071088 ecr 66984834], length 0
04:42:57.834938 IP 127.0.0.1.56877 > 127.0.0.1.5000: Flags [.], ack 37, win 320, options [nop,
nop,TS val 1567613333 ecr 1979855593], length 0
04:42:57.909526 IP 127.0.0.1.14353 > 127.0.0.1.5000: Flags [P.], seq 39:54, ack 39, win 320, o
ptions [nop,nop,TS val 1015181088 ecr 105272309], length 15
04:42:57.909782 IP 127.0.0.1.56877 > 127.0.0.1.5000: Flags [P.], seq 41:57, ack 37, win 320, o
ptions [nop,nop,TS val 1567613408 ecr 1979855593], length 16
04:42:57.909791 IP 127.0.0.1.5000 > 127.0.0.1.14353: Flags [P.], seq 39:56, ack 54, win 320, o
ptions [nop,nop,TS val 105272445 ecr 1015181088], length 17
04:42:57.909820 IP 127.0.0.1.5000 > 127.0.0.1.56877: Flags [P.], seq 37:70, ack 57, win 320, o
ptions [nop,nop,TS val 1979855729 ecr 1567613408], length 33
04:42:57.909832 IP 127.0.0.1.5000 > 127.0.0.1.49609: Flags [P.], seq 21:38, ack 16, win 320, o
ptions [nop,nop,TS val 66984970 ecr 2066071088], length 17
04:42:57.909891 IP 127.0.0.1.5000 > 127.0.0.1.14353: Flags [P.], seq 56:74, ack 54, win 320, o
ptions [nop,nop,TS val 105272445 ecr 1015181088], length 18
04:42:57.909898 IP 127.0.0.1.14353 > 127.0.0.1.5000: Flags [.], ack 74, win 320, options [nop,
nop,TS val 1015181088 ecr 105272445], length 0
04:42:57.909911 IP 127.0.0.1.5000 > 127.0.0.1.25024: Flags [P.], seq 21:38, ack 16, win 320, o
ptions [nop,nop,TS val 3163229235 ecr 972531797], length 17
04:42:57.957549 IP 127.0.0.1.25024 > 127.0.0.1.5000: Flags [.], ack 38, win 320, options [nop,
nop,TS val 972531913 ecr 3163229235], length 0
04:42:57.957555 IP 127.0.0.1.49609 > 127.0.0.1.5000: Flags [.], ack 38, win 320, options [nop,
nop,TS val 2066071204 ecr 66984970], length 0
04:42:57.957557 IP 127.0.0.1.56877 > 127.0.0.1.5000: Flags [.], ack 70, win 320, options [nop,
nop,TS val 1567613449 ecr 1979855729], length 0
04:42:58.046034 IP 127.0.0.1.14353 > 127.0.0.1.5000: Flags [P.], seq 54:69, ack 74, win 320, o
ptions [nop,nop,TS val 1015181218 ecr 105272445], length 15
```

Figure 15: `tcpdump` (separate trade-saturated run) — the server keeps pushing market-data segments to every client and the reading clients keep acknowledging them with an advancing sequence number

Answer. Because the slow client never reads, the `TRADE` data the server sends it accumulates in that client's **TCP receive buffer** — `netstat` shows its `Recv-Q` rising from 850 to about 7000 bytes over 14 seconds, while every other connection stays at 0. The server's own send queue to that client stays small, so the server is not stuck on it. If the slow client kept not reading, its receive window would eventually reach 0, the server's `send()` to that socket would return `EAGAIN`, and the server would hold the unsent bytes in that connection's application

output buffer (capped at 8 MiB, after which that one connection is dropped) — it would still never block. The normal client and the traders were completely unaffected throughout: one stalled reader is isolated to its own connection and does not slow the server for anyone else. This is the point of the non-blocking `poll()` design.

Experiment 8 — Unexpected Client Disconnection

Run: `python3 experiment.py 8`

Question. When a client disappears unexpectedly while the Exchange Server is communicating with it, what happens to their TCP connection, and what network-level evidence allows you to determine how the server detects the failure?

Approach. A separate Market-Data **process** subscribes to `JNST` and stays connected while trades flow to it; a second Market-Data connection stays up for comparison. The first process is then `SIGKILL`ed. Capture the packets around the kill and compare the two connections before and after.

Commands / tools.

```
tcpdump -i lo0 -n 'tcp port 5000'          # FIN/RST from the killed client's port
sockstat -4 | grep ':5000'                 # dead conn disappears, survivor stays
procstat -f $(pgrep exchange_server) | wc -l   # server fd count drops by one
```

Observation. The harness reported the four connections (server PID 7967, `event loop: kqueue`):

```
Surviving Market-Data Client: connected from 127.0.0.1:48671
Buyer Trader:                 connected from 127.0.0.1:45828
Seller Trader:                connected from 127.0.0.1:36484
Disappearing Market-Data Client (helper process): 127.0.0.1:63049
```

```
root@COL334 Assignment:~/COL334-A2 # python3 experiment.py 8
=== Experiment 8: Unexpected Client Disconnection ===

Started Exchange Server (PID 7967).
fd limit (soft): 200000
Exchange Server listening on 127.0.0.1:5000 (event loop: kqueue)
Exchange Server is listening.

Surviving Market-Data Client: connected from 127.0.0.1:48671
Buyer Trader: connected from 127.0.0.1:45828
Seller Trader: connected from 127.0.0.1:36484

Both Market-Data Clients should now be connected.
Generating traffic before the disconnection...


The Market-Data Client process will now disappear unexpectedly.
The client process has disappeared.
Generating additional traffic.

Investigate the terminated connection and the surviving one.
Press Ctrl-C when you are finished.

Experiment finished.
Stopping Exchange Server...
root@COL334 Assignment:~/COL334-A2 #
```

Figure 16: Experiment 8 harness output

Before the kill. `sockstat -4` showed the server holding four client sockets plus the listener — FD 6 → `:48671`, FD 7 → `:45828`, FD 8 → `:36484`, FD 9 → `:63049` (the helper, owned by `python3.12` PID 7970). `tcpdump` showed the server continuously pushing data segments (`Flags [P.]`, `TRADE/BOUGHT/SOLD` payloads of 17–35 bytes) to all clients, each acknowledged — so the server was actively communicating with the helper at the moment it died.

After the SIGKILL. `procstat -f 7967` and `netstat -an -p tcp` both showed the `:63049` connection completely gone: the server now held only FD 5 (listener, `127.0.0.1:5000 *:0`), FD 6 `:48671`, FD 7 `:45828`, FD 8 `:36484`. `netstat` listed the three surviving pairs `ESTABLISHED` plus the `LISTEN` row and nothing for `:63049`. The server's open-fd count dropped by exactly one; it did not crash, and the surviving subscriber `:48671` kept receiving `TRADE` pushes through the post-disconnect traffic. At the end of the run the three remaining clients closed with a normal FIN four-way handshake (`Flags [F.]` from `:45828`, `:36484`, `:48671` at `07:22:23`), confirming the survivors were healthy the whole time. 716 packets captured, 0 dropped by the kernel.

```
[root@COL334 Assignment:~/COL334-A2 # sockstat -4 -c | grep 5000                                    ]
 root exchange_s 7934  6 tcp4  127.0.0.1:5000        127.0.0.1:31784
 root exchange_s 7934  7 tcp4  127.0.0.1:5000        127.0.0.1:52557
 root exchange_s 7934  8 tcp4  127.0.0.1:5000        127.0.0.1:29779
 root python3.12 7933  3 tcp4  127.0.0.1:31784       127.0.0.1:5000
 root python3.12 7933  4 tcp4  127.0.0.1:52557       127.0.0.1:5000
 root python3.12 7933  5 tcp4  127.0.0.1:29779       127.0.0.1:5000
[root@COL334 Assignment:~/COL334-A2 # sockstat -4 -c | grep 5000                                    ]
 root python3.12 7970  3 tcp4  127.0.0.1:63049       127.0.0.1:5000
 root exchange_s 7967  6 tcp4  127.0.0.1:5000        127.0.0.1:48671
 root exchange_s 7967  7 tcp4  127.0.0.1:5000        127.0.0.1:45828
 root exchange_s 7967  8 tcp4  127.0.0.1:5000        127.0.0.1:36484
 root exchange_s 7967  9 tcp4  127.0.0.1:5000        127.0.0.1:63049
 root python3.12 7966  3 tcp4  127.0.0.1:48671       127.0.0.1:5000
 root python3.12 7966  4 tcp4  127.0.0.1:45828       127.0.0.1:5000
 root python3.12 7966  5 tcp4  127.0.0.1:36484       127.0.0.1:5000
[root@COL334 Assignment:~/COL334-A2 # ps aux | grep '[e]xchange_server'                              ]
 root  7967   0.2  0.1 14548  3604  -  Ss   07:22      0:00.01 /root/COL334-A2/exchange_server 127.0.0.1 50
[root@COL334 Assignment:~/COL334-A2 # pgrep exchange_server                                          ]
 7967
[root@COL334 Assignment:~/COL334-A2 # procstat -f 7967                                               ]
   PID COMM                FD T V FLAGS    REF  OFFSET PRO NAME
  7967 exchange_server   text v r r-------   -       - -   /root/COL334-A2/exchange_server
  7967 exchange_server    cwd v d r-------   -       - -   /root/COL334-A2
  7967 exchange_server   root v d r-------   -       - -   /
  7967 exchange_server      0 v c rw------   9   38633 -   /dev/pts/0
  7967 exchange_server      1 v c rw------   9   38633 -   /dev/pts/0
  7967 exchange_server      2 v c rw------   9   38633 -   /dev/pts/0
  7967 exchange_server      3 k - rw------   2       0 -   -
  7967 exchange_server      4 v c r-------   1       0 -   /dev/null
  7967 exchange_server      5 s - rw---n--   2       0 TCP 0 0 127.0.0.1:5000 *:0
  7967 exchange_server      6 s - rw---n--   2       0 TCP 0 0 127.0.0.1:5000 127.0.0.1:48671
  7967 exchange_server      7 s - rw---n--   2       0 TCP 0 0 127.0.0.1:5000 127.0.0.1:45828
  7967 exchange_server      8 s - rw---n--   2       0 TCP 0 0 127.0.0.1:5000 127.0.0.1:36484
[root@COL334 Assignment:~/COL334-A2 # netstat -an -p tcp | grep 5000                                 ]
 tcp4           0      0 127.0.0.1.5000         127.0.0.1.36484        ESTABLISHED
 tcp4           0      0 127.0.0.1.36484        127.0.0.1.5000         ESTABLISHED
 tcp4           0      0 127.0.0.1.5000         127.0.0.1.45828        ESTABLISHED
 tcp4           0      0 127.0.0.1.45828        127.0.0.1.5000         ESTABLISHED
 tcp4           0      0 127.0.0.1.5000         127.0.0.1.48671        ESTABLISHED
 tcp4           0      0 127.0.0.1.48671        127.0.0.1.5000         ESTABLISHED
 tcp4           0      0 127.0.0.1.5000         *.*                    LISTEN
 root@COL334 Assignment:~/COL334-A2 #
```

Figure 17: sockets before vs after the kill

```
07:22:07.840675 IP 127.0.0.1.45828 > 127.0.0.1.5000: Flags [P.], seq 1048635304:1048635319, ack 657382812,
win 320, options [nop,nop,TS val 1191451013 ecr 2925148466], length 15
07:22:07.840726 IP 127.0.0.1.36484 > 127.0.0.1.5000: Flags [P.], seq 3814585409:3814585425, ack 3859230815,
 win 320, options [nop,nop,TS val 3605796380 ecr 378105564], length 16
07:22:07.840873 IP 127.0.0.1.5000 > 127.0.0.1.45828: Flags [P.], seq 657382812:657382831, ack 1048635319, w
in 320, options [nop,nop,TS val 2925148559 ecr 1191451013], length 19
07:22:07.840902 IP 127.0.0.1.5000 > 127.0.0.1.36484: Flags [P.], seq 3859230815:3859230850, ack 3814585425,
 win 320, options [nop,nop,TS val 378105657 ecr 3605796380], length 35
07:22:07.840918 IP 127.0.0.1.5000 > 127.0.0.1.45828: Flags [P.], seq 657382831:657382849, ack 1048635319, w
in 320, options [nop,nop,TS val 2925148559 ecr 1191451013], length 18
07:22:07.840921 IP 127.0.0.1.5000 > 127.0.0.1.48671: Flags [P.], seq 3387042649:3387042666, ack 581862015,
win 320, options [nop,nop,TS val 2811191904 ecr 4209211385], length 17
07:22:07.840974 IP 127.0.0.1.45828 > 127.0.0.1.5000: Flags [.], ack 657382849, win 320, options [nop,nop,TS
 val 1191451013 ecr 2925148559], length 0
07:22:07.893776 IP 127.0.0.1.48671 > 127.0.0.1.5000: Flags [.], ack 3387042666, win 320, options [nop,nop,T
S val 4209211493 ecr 2811191904], length 0
07:22:07.893802 IP 127.0.0.1.36484 > 127.0.0.1.5000: Flags [.], ack 3859230850, win 320, options [nop,nop,T
S val 3605796434 ecr 378105657], length 0
07:22:07.930044 IP 127.0.0.1.45828 > 127.0.0.1.5000: Flags [P.], seq 1048635319:1048635334, ack 657382849,
win 320, options [nop,nop,TS val 1191451103 ecr 2925148559], length 15
07:22:07.930106 IP 127.0.0.1.36484 > 127.0.0.1.5000: Flags [P.], seq 3814585425:3814585441, ack 3859230850,
 win 320, options [nop,nop,TS val 3605796470 ecr 378105657], length 16
07:22:07.930265 IP 127.0.0.1.5000 > 127.0.0.1.45828: Flags [P.], seq 657382849:657382868, ack 1048635334, w
in 320, options [nop,nop,TS val 2925148649 ecr 1191451103], length 19
07:22:07.930290 IP 127.0.0.1.5000 > 127.0.0.1.36484: Flags [P.], seq 3859230850:3859230885, ack 3814585441,
 win 320, options [nop,nop,TS val 378105747 ecr 3605796470], length 35
07:22:07.930302 IP 127.0.0.1.5000 > 127.0.0.1.45828: Flags [P.], seq 657382868:657382886, ack 1048635334, w
in 320, options [nop,nop,TS val 2925148649 ecr 1191451103], length 18
07:22:07.930305 IP 127.0.0.1.5000 > 127.0.0.1.48671: Flags [P.], seq 3387042666:3387042683, ack 581862015,
win 320, options [nop,nop,TS val 2811191994 ecr 4209211493], length 17
07:22:07.930327 IP 127.0.0.1.45828 > 127.0.0.1.5000: Flags [.], ack 657382886, win 320, options [nop,nop,TS
 val 1191451103 ecr 2925148649], length 0
07:22:07.977238 IP 127.0.0.1.48671 > 127.0.0.1.5000: Flags [.], ack 3387042683, win 320, options [nop,nop,T
S val 4209211576 ecr 2811191994], length 0
07:22:07.977253 IP 127.0.0.1.36484 > 127.0.0.1.5000: Flags [.], ack 3859230885, win 320, options [nop,nop,T
S val 3605796517 ecr 378105747], length 0
07:22:08.022303 IP 127.0.0.1.45828 > 127.0.0.1.5000: Flags [P.], seq 1048635334:1048635349, ack 657382886,
win 320, options [nop,nop,TS val 1191451195 ecr 2925148649], length 15
07:22:08.022367 IP 127.0.0.1.36484 > 127.0.0.1.5000: Flags [P.], seq 3814585441:3814585457, ack 3859230885,
 win 320, options [nop,nop,TS val 3605796562 ecr 378105747], length 16
07:22:08.022526 IP 127.0.0.1.5000 > 127.0.0.1.45828: Flags [P.], seq 657382886:657382905, ack 1048635349, w
in 320, options [nop,nop,TS val 2925148741 ecr 1191451195], length 19
07:22:08.022559 IP 127.0.0.1.5000 > 127.0.0.1.36484: Flags [P.], seq 3859230885:3859230920, ack 3814585457,
 win 320, options [nop,nop,TS val 378105839 ecr 3605796562], length 35
07:22:08.022573 IP 127.0.0.1.5000 > 127.0.0.1.45828: Flags [P.], seq 657382905:657382923, ack 1048635349, w
in 320, options [nop,nop,TS val 2925148741 ecr 1191451195], length 18
07:22:08.022577 IP 127.0.0.1.5000 > 127.0.0.1.48671: Flags [P.], seq 3387042683:3387042700, ack 581862015,
win 320, options [nop,nop,TS val 2811192086 ecr 4209211576], length 17
07:22:08.022619 IP 127.0.0.1.45828 > 127.0.0.1.5000: Flags [.], ack 657382923, win 320, options [nop,nop,TS
 val 1191451195 ecr 2925148741], length 0
07:22:08.059385 IP 127.0.0.1.48671 > 127.0.0.1.5000: Flags [.], ack 3387042700, win 320, options [nop,nop,T
S val 4209211658 ecr 2811192086], length 0
07:22:08.059400 IP 127.0.0.1.36484 > 127.0.0.1.5000: Flags [.], ack 3859230920, win 320, options [nop,nop,T
S val 3605796599 ecr 378105839], length 0
07:22:08.111205 IP 127.0.0.1.45828 > 127.0.0.1.5000: Flags [P.], seq 1048635349:1048635364, ack 657382923,
win 320, options [nop,nop,TS val 1191451284 ecr 2925148741], length 15
07:22:08.111270 IP 127.0.0.1.36484 > 127.0.0.1.5000: Flags [P.], seq 3814585457:3814585473, ack 3859230920,
 win 320, options [nop,nop,TS val 3605796651 ecr 378105839], length 16
```

Figure 18: tcpdump — server pushing data segments to all three clients before the kill

The kill itself. `tcpdump` caught the exact sequence on the helper's port `:63049`:

Time	Segment	Flags	Meaning
`07:22:03.994158`	`:5000` → `:63049` len 17	`[P.]`	server still actively pushing market data to the helper
`07:22:04.130229`	`:63049` → `:5000`	`[F.]`	the killed process's kernel sends a **FIN** (seq 3523400548)
`07:22:04.130293`	`:5000` → `:63049`	`[.]`	server ACKs the FIN — `recv()` returned 0
`07:22:04.130517`	`:5000` → `:63049`	`[F.]`	server closes its own end, sends its FIN
`07:22:04.130536`	`:63049` → `:5000`	`[.]`	final ACK — connection fully closed

Immediately after (`07:22:04.134…`) the three surviving connections (`:45828`, `:36484`, `:48671`) are still exchanging `[P.]` data segments normally — the kill of `:63049` did not perturb them.

```
07:22:03.890445 IP 127.0.0.1.36484 > 127.0.0.1.5000: Flags [.], ack 3859229341, win 320, options [nop,nop,T
S val 3605792430 ecr 378101669], length 0
07:22:03.994030 IP 127.0.0.1.45828 > 127.0.0.1.5000: Flags [P.], seq 1048634659:1048634674, ack 657381252,
win 320, options [nop,nop,TS val 1191447167 ecr 2925144571], length 15
07:22:03.994057 IP 127.0.0.1.36484 > 127.0.0.1.5000: Flags [P.], seq 3814584721:3814584737, ack 3859229341,
 win 320, options [nop,nop,TS val 3605792534 ecr 378101669], length 16
07:22:03.994127 IP 127.0.0.1.5000 > 127.0.0.1.45828: Flags [P.], seq 657381252:657381270, ack 1048634674, w
in 320, options [nop,nop,TS val 2925144713 ecr 1191447167], length 18
07:22:03.994147 IP 127.0.0.1.5000 > 127.0.0.1.36484: Flags [P.], seq 3859229341:3859229375, ack 3814584737,
 win 320, options [nop,nop,TS val 378101811 ecr 3605792534], length 34
07:22:03.994156 IP 127.0.0.1.5000 > 127.0.0.1.45828: Flags [P.], seq 657381270:657381288, ack 1048634674, w
in 320, options [nop,nop,TS val 2925144713 ecr 1191447167], length 18
07:22:03.994158 IP 127.0.0.1.5000 > 127.0.0.1.63049: Flags [P.], seq 849029139:849029156, ack 3523400548, w
in 320, options [nop,nop,TS val 3349849358 ecr 3881499734], length 17
07:22:03.994160 IP 127.0.0.1.5000 > 127.0.0.1.48671: Flags [P.], seq 3387041918:3387041935, ack 581862015,
win 320, options [nop,nop,TS val 2811188058 ecr 4209207489], length 17
07:22:03.994176 IP 127.0.0.1.45828 > 127.0.0.1.5000: Flags [.], ack 657381288, win 320, options [nop,nop,TS
 val 1191447167 ecr 2925144713], length 0
07:22:04.035911 IP 127.0.0.1.48671 > 127.0.0.1.5000: Flags [.], ack 3387041935, win 320, options [nop,nop,T
S val 4209207628 ecr 2811188058], length 0
07:22:04.035971 IP 127.0.0.1.63049 > 127.0.0.1.5000: Flags [.], ack 849029156, win 320, options [nop,nop,TS
 val 3881499873 ecr 3349849358], length 0
07:22:04.035972 IP 127.0.0.1.36484 > 127.0.0.1.5000: Flags [.], ack 3859229375, win 320, options [nop,nop,T
S val 3605792569 ecr 378101811], length 0
07:22:04.130229 IP 127.0.0.1.63049 > 127.0.0.1.5000: Flags [F.], seq 3523400548, ack 849029156, win 320, op
tions [nop,nop,TS val 3881499974 ecr 3349849358], length 0
07:22:04.130293 IP 127.0.0.1.5000 > 127.0.0.1.63049: Flags [.], ack 3523400549, win 320, options [nop,nop,T
S val 3349849494 ecr 3881499974], length 0
07:22:04.130517 IP 127.0.0.1.5000 > 127.0.0.1.63049: Flags [F.], seq 849029156, ack 3523400549, win 320, op
tions [nop,nop,TS val 3349849494 ecr 3881499974], length 0
07:22:04.130536 IP 127.0.0.1.63049 > 127.0.0.1.5000: Flags [.], ack 849029157, win 320, options [nop,nop,TS
 val 3881499974 ecr 3349849494], length 0
07:22:04.134205 IP 127.0.0.1.45828 > 127.0.0.1.5000: Flags [P.], seq 1048634674:1048634689, ack 657381288,
win 320, options [nop,nop,TS val 1191447303 ecr 2925144713], length 15
07:22:04.134236 IP 127.0.0.1.36484 > 127.0.0.1.5000: Flags [P.], seq 3814584737:3814584753, ack 3859229375,
 win 320, options [nop,nop,TS val 3605792670 ecr 378101811], length 16
07:22:04.134283 IP 127.0.0.1.5000 > 127.0.0.1.45828: Flags [P.], seq 657381288:657381306, ack 1048634689, w
in 320, options [nop,nop,TS val 2925144849 ecr 1191447303], length 18
07:22:04.134313 IP 127.0.0.1.5000 > 127.0.0.1.36484: Flags [P.], seq 3859229375:3859229409, ack 3814584753,
 win 320, options [nop,nop,TS val 378101947 ecr 3605792670], length 34
07:22:04.134325 IP 127.0.0.1.5000 > 127.0.0.1.45828: Flags [P.], seq 657381306:657381324, ack 1048634689, w
in 320, options [nop,nop,TS val 2925144849 ecr 1191447303], length 18
07:22:04.134328 IP 127.0.0.1.5000 > 127.0.0.1.48671: Flags [P.], seq 3387041935:3387041952, ack 581862015,
win 320, options [nop,nop,TS val 2811188194 ecr 4209207628], length 17
07:22:04.134702 IP 127.0.0.1.45828 > 127.0.0.1.5000: Flags [.], ack 657381324, win 320, options [nop,nop,TS
 val 1191447303 ecr 2925144849], length 0
07:22:04.175990 IP 127.0.0.1.48671 > 127.0.0.1.5000: Flags [.], ack 3387041952, win 320, options [nop,nop,T
S val 4209207775 ecr 2811188194], length 0
07:22:04.176015 IP 127.0.0.1.36484 > 127.0.0.1.5000: Flags [.], ack 3859229409, win 320, options [nop,nop,T
```

Figure 19: tcpdump — the FIN four-way close on the killed helper's port :63049, survivors still exchanging data

```
 win 320, options [nop,nop,TS val 378106209 ecr 3605796932], length 35
07:22:08.392852 IP 127.0.0.1.5000 > 127.0.0.1.45828: Flags [P.], seq 657383053:657383071, ack 1048635409, w
in 320, options [nop,nop,TS val 2925149111 ecr 1191451565], length 18
07:22:08.392856 IP 127.0.0.1.5000 > 127.0.0.1.48671: Flags [P.], seq 3387042751:3387042768, ack 581862015,
win 320, options [nop,nop,TS val 2811192456 ecr 4209211970], length 17
07:22:08.392893 IP 127.0.0.1.45828 > 127.0.0.1.5000: Flags [.], ack 657383071, win 320, options [nop,nop,TS
 val 1191451565 ecr 2925149111], length 0
07:22:08.430034 IP 127.0.0.1.48671 > 127.0.0.1.5000: Flags [.], ack 3387042768, win 320, options [nop,nop,T
S val 4209212029 ecr 2811192456], length 0
07:22:08.430052 IP 127.0.0.1.36484 > 127.0.0.1.5000: Flags [.], ack 3859231060, win 320, options [nop,nop,T
S val 3605796970 ecr 378106209], length 0
07:22:08.480846 IP 127.0.0.1.45828 > 127.0.0.1.5000: Flags [P.], seq 1048635409:1048635424, ack 657383071,
win 320, options [nop,nop,TS val 1191451654 ecr 2925149111], length 15
07:22:08.480951 IP 127.0.0.1.36484 > 127.0.0.1.5000: Flags [P.], seq 3814585521:3814585537, ack 3859231060,
 win 320, options [nop,nop,TS val 3605797021 ecr 378106209], length 16
07:22:08.480993 IP 127.0.0.1.5000 > 127.0.0.1.45828: Flags [P.], seq 657383071:657383090, ack 1048635424, w
in 320, options [nop,nop,TS val 2925149200 ecr 1191451654], length 19
07:22:08.481036 IP 127.0.0.1.5000 > 127.0.0.1.36484: Flags [P.], seq 3859231060:3859231095, ack 3814585537,
 win 320, options [nop,nop,TS val 378106298 ecr 3605797021], length 35
07:22:08.481045 IP 127.0.0.1.5000 > 127.0.0.1.45828: Flags [P.], seq 657383090:657383108, ack 1048635424, w
in 320, options [nop,nop,TS val 2925149200 ecr 1191451654], length 18
07:22:08.481049 IP 127.0.0.1.5000 > 127.0.0.1.48671: Flags [P.], seq 3387042768:3387042785, ack 581862015,
win 320, options [nop,nop,TS val 2811192545 ecr 4209212029], length 17
07:22:08.481052 IP 127.0.0.1.45828 > 127.0.0.1.5000: Flags [.], ack 657383108, win 320, options [nop,nop,TS
 val 1191451654 ecr 2925149200], length 0
07:22:08.534823 IP 127.0.0.1.48671 > 127.0.0.1.5000: Flags [.], ack 3387042785, win 320, options [nop,nop,T
S val 4209212134 ecr 2811192545], length 0
07:22:08.534851 IP 127.0.0.1.36484 > 127.0.0.1.5000: Flags [.], ack 3859231095, win 320, options [nop,nop,T
S val 3605797075 ecr 378106298], length 0
07:22:23.568983 IP 127.0.0.1.48671 > 127.0.0.1.5000: Flags [F.], seq 581862015, ack 3387042785, win 320, op
tions [nop,nop,TS val 4209227167 ecr 2811192545], length 0
07:22:23.569031 IP 127.0.0.1.5000 > 127.0.0.1.48671: Flags [.], ack 581862016, win 320, options [nop,nop,TS
 val 2811207632 ecr 4209227167], length 0
07:22:23.569085 IP 127.0.0.1.45828 > 127.0.0.1.5000: Flags [F.], seq 1048635424, ack 657383108, win 320, op
tions [nop,nop,TS val 1191466741 ecr 2925149200], length 0
07:22:23.569096 IP 127.0.0.1.5000 > 127.0.0.1.45828: Flags [.], ack 1048635425, win 320, options [nop,nop,T
S val 2925164287 ecr 1191466741], length 0
07:22:23.569117 IP 127.0.0.1.36484 > 127.0.0.1.5000: Flags [F.], seq 3814585537, ack 3859231095, win 320, o
ptions [nop,nop,TS val 3605812108 ecr 378106298], length 0
07:22:23.569131 IP 127.0.0.1.5000 > 127.0.0.1.36484: Flags [.], ack 3814585538, win 320, options [nop,nop,T
S val 378121385 ecr 3605812108], length 0
07:22:23.569521 IP 127.0.0.1.5000 > 127.0.0.1.36484: Flags [F.], seq 3859231095, ack 3814585538, win 320, o
ptions [nop,nop,TS val 378121386 ecr 3605812108], length 0
07:22:23.569539 IP 127.0.0.1.36484 > 127.0.0.1.5000: Flags [.], ack 3859231096, win 320, options [nop,nop,T
S val 3605812109 ecr 378121386], length 0
07:22:23.569682 IP 127.0.0.1.5000 > 127.0.0.1.48671: Flags [F.], seq 3387042785, ack 581862016, win 320, op
tions [nop,nop,TS val 2811207633 ecr 4209227167], length 0
07:22:23.569744 IP 127.0.0.1.48671 > 127.0.0.1.5000: Flags [.], ack 3387042786, win 320, options [nop,nop,T
S val 4209227168 ecr 2811207633], length 0
07:22:23.569752 IP 127.0.0.1.5000 > 127.0.0.1.45828: Flags [F.], seq 657383108, ack 1048635425, win 320, op
tions [nop,nop,TS val 2925164288 ecr 1191466741], length 0
07:22:23.569754 IP 127.0.0.1.45828 > 127.0.0.1.5000: Flags [.], ack 657383109, win 320, options [nop,nop,TS
 val 1191466742 ecr 2925164288], length 0
^C
716 packets captured
716 packets received by filter
0 packets dropped by kernel
```

Figure 20: tcpdump — the three survivors' orderly FIN four-way close at the end of the run, 716 packets, 0 dropped

Answer. When the helper process is `SIGKILL`ed it never runs its own shutdown code, but the OS still closes its descriptors, and here the kernel sent a normal **FIN** — captured at `07:22:04.130229` from `127.0.0.1:63049`, followed by the full four-way close (server ACK, server FIN, client ACK). (A **RST** would appear instead if unacknowledged data were still queued toward the dead socket at the instant of the kill; in this run the server's send side to `:63049` had drained, so a clean FIN was sent.) TCP has no other "client gone" signal, so the server learns of the failure only by doing I/O: its `kqueue` loop reports `EVFILT_READ` with `EV_EOF` (equivalently

POLLHUP/POLLERR under poll), or the next send() fails with EPIPE/ECONNRESET, or recv() returns 0. Here recv() returned 0, and the server then close_client()s that connection: it is removed from the poller and the subscriber table, its descriptor is closed, and it disappears from sockstat, procstat, and netstat — the fd count drops by exactly one. The **surviving** Market-Data connection and the two traders were completely unaffected and kept exchanging data throughout. The network evidence is the lone FIN/close exchange on :63049 in the capture together with that connection vanishing from the socket table while every other connection stayed ESTABLISHED.

Bonus — Connection Scalability and I/O Design (§6.9)

Setup. As root, raise the host limits and add loopback aliases (one source IP has only ~64k ephemeral ports to a single destination), then build the kqueue server:

```
sysctl kern.maxfiles=1000000 kern.maxfilesperproc=900000
sysctl kern.ipc.somaxconn=1024 kern.ipc.maxsockets=1000000
for i in 2 3 4 5 6 7 8 9 10; do ifconfig lo0 alias 127.0.0.$i/8; done
make clean && make POLLER=kqueue
./server/run-server 127.0.0.1 5000
```

In a second shell (limit descriptors 200000 in tcsh, or ulimit -n 200000 in sh), open and hold N idle connections:

```
python3 tests/bench/connflood.py 127.0.0.1 5000 <N> --ramp 10000 --src-aliases 9
```

It prints a line every 10,000 connections — pause there and record a table row.

Design note. The server maintains **no thread or process per connection** — it multiplexes all sockets in one kqueue loop. Per-connection cost is therefore just the socket, a file descriptor, and ~0.3 KB of application state; there is no per-connection stack or scheduler entry.

Measurement. A single snap.sh helper recorded, at each 10 000-connection step: the timestamp and server PID; the established-connection count from netstat (on loopback this counts **both** endpoints of every connection, so it reads 2N + 1 including the listener); the server's open FD count from procstat; RSS and %CPU from ps; kern.openfiles / kern.maxfiles from sysctl; and the mbuf / socket-buffer cluster usage from netstat -m.

Resource-measurement table (FreeBSD VM, kqueue build, server PID 20939)

Idle connections (N)	Server RSS	Server %CPU	Server open FDs	netstat ESTABLISHED (both ends)	System-wide open files (kern.openfiles / kern.maxfiles)	Socket-buffer clusters in use / max (netstat -m)
10 000	32 792 KB	0.1 %	10 011	20 001	20 134 / 1 000 000	0 / 262 144
20 000	32 976 KB	0.6 %	20 011	40 001	40 134 / 1 000 000	0 / 262 144
30 000	33 152 KB	0.4 %	30 011	60 001	60 134 / 1 000 000	0 / 262 144
40 000	33 328 KB	0.5 %	40 011	80 001	80 134 / 1 000 000	0 / 262 144
50 000	37 932 KB	0.5 %	50 011	100 001	100 134 / 1 000 000	0 / 262 144
60 000	42 812 KB	0.1 %	60 011	120 001	120 134 / 1 000 000	0 / 262 144

Idle connections (N)	Server RSS	Server %CPU	Server open FDs	`netstat` ESTABLISHED (both ends)	System-wide open files (`kern.openfiles` / `kern.maxfiles`)	Socket-buffer clusters in use / max (`netstat -m`)
70 000	43 888 KB	0.0 %	70 011	140 001	140 134 / 1 000 000	0 / 262 144

From 10 000 to 70 000 idle connections the server's FD count tracks N exactly (N + 11 for stdio, the listener, the `kqueue` fd, etc.), RSS grows by only ~11 MB total (≈0.16 KB per extra connection), `%CPU` never leaves the 0–0.6 % noise band, and the mbuf-cluster count stays at **0** — the kernel allocates no socket-buffer memory for a connection with nothing queued, because the server does not pin `SO_SNDBUF`/`SO_RCVBUF`.

```
root@COL334 Assignment:~/COL334-A2 # /root/snap.sh
=== SNAPSHOT Tue Sep  8 08:26:33 IST 2026 ===
PID: 20939
-- established connections --
20001
-- open file descriptors --
   10011
-- memory/cpu --
  RSS %CPU
32792  0.1
-- system fd usage --
kern.openfiles: 20134
kern.maxfiles: 1000000
-- mbuf usage --
577/3743/4320 mbufs in use (current/cache/total)
0/1270/1270/262144 mbuf clusters in use (current/cache/total/max)
0/1270 mbuf+clusters out of packet secondary zone in use (current/cache)
320/1204/1524/127425 4k (page size) jumbo clusters in use (current/cache/total/max)
0/0/0/37755 9k jumbo clusters in use (current/cache/total/max)
=== END ===
root@COL334 Assignment:~/COL334-A2 # /root/snap.sh
=== SNAPSHOT Tue Sep  8 08:27:28 IST 2026 ===
PID: 20939
-- established connections --
40001
-- open file descriptors --
   20011
-- memory/cpu --
  RSS %CPU
32976  0.6
-- system fd usage --
kern.openfiles: 40134
kern.maxfiles: 1000000
-- mbuf usage --
577/3743/4320 mbufs in use (current/cache/total)
0/1270/1270/262144 mbuf clusters in use (current/cache/total/max)
0/1270 mbuf+clusters out of packet secondary zone in use (current/cache)
320/1204/1524/127425 4k (page size) jumbo clusters in use (current/cache/total/max)
0/0/0/37755 9k jumbo clusters in use (current/cache/total/max)
=== END ===
```

Figure 21: `snap.sh` at 10 000 and 20 000 idle connections

```
root@COL334 Assignment:~/COL334-A2 # /root/snap.sh
=== SNAPSHOT Tue Sep  8 08:29:00 IST 2026 ===
PID: 20939
-- established connections --
60001
-- open file descriptors --
   30011
-- memory/cpu --
  RSS %CPU
33152  0.4
-- system fd usage --
kern.openfiles: 60134
kern.maxfiles: 1000000
-- mbuf usage --
577/3743/4320 mbufs in use (current/cache/total)
0/1270/1270/262144 mbuf clusters in use (current/cache/total/max)
0/1270 mbuf+clusters out of packet secondary zone in use (current/cache)
320/1204/1524/127425 4k (page size) jumbo clusters in use (current/cache/total/max)
0/0/0/37755 9k jumbo clusters in use (current/cache/total/max)
=== END ===
root@COL334 Assignment:~/COL334-A2 # /root/snap.sh
=== SNAPSHOT Tue Sep  8 08:30:34 IST 2026 ===
PID: 20939
-- established connections --
80001
-- open file descriptors --
   40011
-- memory/cpu --
  RSS %CPU
33328  0.5
-- system fd usage --
kern.openfiles: 80134
kern.maxfiles: 1000000
-- mbuf usage --
577/3743/4320 mbufs in use (current/cache/total)
0/1270/1270/262144 mbuf clusters in use (current/cache/total/max)
0/1270 mbuf+clusters out of packet secondary zone in use (current/cache)
320/1204/1524/127425 4k (page size) jumbo clusters in use (current/cache/total/max)
0/0/0/37755 9k jumbo clusters in use (current/cache/total/max)
=== END ===
root@COL334 Assignment:~/COL334-A2 # /root/snap.sh
=== SNAPSHOT Tue Sep  8 08:32:14 IST 2026 ===
PID: 20939
-- established connections --
100001
-- open file descriptors --
   50011
-- memory/cpu --
  RSS %CPU
37932  0.5
-- system fd usage --
kern.openfiles: 100134
kern.maxfiles: 1000000
-- mbuf usage --
577/3743/4320 mbufs in use (current/cache/total)
0/1270/1270/262144 mbuf clusters in use (current/cache/total/max)
0/1270 mbuf+clusters out of packet secondary zone in use (current/cache)
320/1204/1524/127425 4k (page size) jumbo clusters in use (current/cache/total/max)
```

Figure 22: `snap.sh` at 30 000, 40 000 and 50 000 idle connections

```
root@COL334 Assignment:~/COL334-A2 # /root/snap.sh
=== SNAPSHOT Tue Sep  8 08:32:14 IST 2026 ===
PID: 20939
-- established connections --
100001
-- open file descriptors --
   50011
-- memory/cpu --
  RSS %CPU
37932  0.5
-- system fd usage --
kern.openfiles: 100134
kern.maxfiles: 1000000
-- mbuf usage --
577/3743/4320 mbufs in use (current/cache/total)
0/1270/1270/262144 mbuf clusters in use (current/cache/total/max)
0/1270 mbuf+clusters out of packet secondary zone in use (current/cache)
320/1204/1524/127425 4k (page size) jumbo clusters in use (current/cache/total/max)
0/0/0/37755 9k jumbo clusters in use (current/cache/total/max)
=== END ===
root@COL334 Assignment:~/COL334-A2 # /root/snap.sh
=== SNAPSHOT Tue Sep  8 08:33:53 IST 2026 ===
PID: 20939
-- established connections --
120001
-- open file descriptors --
   60011
-- memory/cpu --
  RSS %CPU
42812  0.1
-- system fd usage --
kern.openfiles: 120134
kern.maxfiles: 1000000
-- mbuf usage --
577/3743/4320 mbufs in use (current/cache/total)
0/1270/1270/262144 mbuf clusters in use (current/cache/total/max)
0/1270 mbuf+clusters out of packet secondary zone in use (current/cache)
320/1204/1524/127425 4k (page size) jumbo clusters in use (current/cache/total/max)
0/0/0/37755 9k jumbo clusters in use (current/cache/total/max)
=== END ===
root@COL334 Assignment:~/COL334-A2 # /root/snap.sh
=== SNAPSHOT Tue Sep  8 08:35:35 IST 2026 ===
PID: 20939
-- established connections --
140001
-- open file descriptors --
   70011
-- memory/cpu --
  RSS %CPU
43888  0.0
-- system fd usage --
kern.openfiles: 140134
kern.maxfiles: 1000000
-- mbuf usage --
577/3743/4320 mbufs in use (current/cache/total)
0/1270/1270/262144 mbuf clusters in use (current/cache/total/max)
```

Figure 23: `snap.sh` at 50 000, 60 000 and 70 000 idle connections

```
root@COL334 Assignment:~/COL334-A2 # /root/snap.sh
=== SNAPSHOT Tue Sep  8 08:35:35 IST 2026 ===
PID: 20939
-- established connections --
140001
-- open file descriptors --
   70011
-- memory/cpu --
  RSS %CPU
43888  0.0
-- system fd usage --
kern.openfiles: 140134
kern.maxfiles: 1000000
-- mbuf usage --
577/3743/4320 mbufs in use (current/cache/total)
0/1270/1270/262144 mbuf clusters in use (current/cache/total/max)
0/1270 mbuf+clusters out of packet secondary zone in use (current/cache)
320/1204/1524/127425 4k (page size) jumbo clusters in use (current/cache/total/max)
0/0/0/37755 9k jumbo clusters in use (current/cache/total/max)
=== END ===
root@COL334 Assignment:~/COL334-A2 #
```

Figure 24: `snap.sh` final reading — 70 000 idle connections held (140 001 endpoints, 70 011 FDs, RSS 43 888 KB, 0.0 %CPU)

Analysis

1. **Does the server maintain all requested connections? Yes** — all 70 000 were established and held simultaneously (`netstat` shows 140 001 endpoints, `procstat` shows 70 011 server FDs), with no connection refused or dropped and the server still responsive. The server raises its own `RLIMIT_NOFILE` to the hard limit at startup, so the ceiling is the system limits (`kern.maxfiles = 1 000 000`, `kern.maxfilesperproc`), not a small inherited soft limit; at 70 000 connections `kern.openfiles` is only 140 134, ~14 % of `kern.maxfiles`, so there is substantial headroom left.

2. **First significant bottleneck.** For the pure *idle-holding* workload nothing in the table saturates — RSS is ~44 MB at 70 000 connections, system FD usage is ~14 % of the limit, socket-buffer clusters are 0, and `%CPU` is ~0 (idle connections generate no readiness events, so the event loop simply sleeps). The bottleneck appears once those connections carry **traffic**, and it is **CPU spent per event-loop wakeup**, scaling differently by primitive:

   - with the portable `poll(2)` build the kernel is handed the whole descriptor array and the server scans all of it on *every* wakeup, so the cost of servicing one active event is **O(N)** in the total connection count — an active client's round-trip latency grows with the number of otherwise-idle connections;
   - with the `kqueue` build the kernel returns only the descriptors that are actually ready, so servicing an active event is **O(ready)**, independent of how many idle connections exist.

   The `kqueue` resource table is consistent with this — `%CPU` never leaves the 0–0.6 % band from 10 000 to 70 000 idle connections — and Q5 measures the `poll` build directly for comparison.

3. **How the concurrency/I/O design contributes.** The single-thread, single readiness-loop, no-thread-per-connection design is what keeps RSS flat (~0.16 KB extra per connection, one scheduler entity, no per-connection stack) and lets one process hold 70 000 sockets at all. The one liability of the design is that the *portable* form of it — `poll(2)` — must re-examine every descriptor each wakeup, making the loop's cost proportional to how many sockets **exist** rather than how many are **active**. Selecting `kqueue` at build time removes that liability without changing anything else about the architecture.

4. **Would changing the mechanism help?** For the resource use in the table, no — Q5 shows the `poll` build holds 70 000 idle connections just as well (and with *less* memory). The mechanism matters for **per-event latency under load**: swapping `poll` for `kqueue` (FreeBSD) or `epoll` (Linux) makes servicing an active event O(ready) instead of O(total), so an active client's latency stops depending on the idle-connection count. Switching the other direction, to **thread-per-connection**, would *not* help: it would

trade the `poll` scan for ~70 000 thread stacks (MBs each — gigabytes total) plus scheduler and lock contention, and would hit a hard wall long before 70 000 connections.

5. **Quantify the trade-off.** We re-ran the same measurement on the default `poll` build holding 70 000 idle connections:

Metric at 70 000 idle	`poll` build	`kqueue` build
Server open FDs (`procstat -f`)	70 011	70 011
Server RSS (`ps -o rss`)	26 196 KB	43 888 KB
Server %CPU (idle)	0.0 %	0.0 %
`kern.openfiles`	140 123	140 134
mbuf / socket-buffer clusters in use	0	0
All 70 000 connections held?	yes	yes

```
root@COL334 Assignment:~/COL334-A2 # make clean && make POLLER=poll
rm -f exchange_server trader_client market_data_client server_test
c++ -O2 -pipe  src/server.cpp -o exchange_server
c++ -O2 -pipe  src/trader.cpp -o trader_client
c++ -O2 -pipe  src/market_data.cpp -o market_data_client
root@COL334 Assignment:~/COL334-A2 # ./server/run-server 127.0.0.1 5000
fd limit (soft): 200000
Exchange Server listening on 127.0.0.1:5000 (event loop: poll)
Terminated
root@COL334 Assignment:~/COL334-A2 # cd ~/COL334-A2
root@COL334 Assignment:~/COL334-A2 # PID=$(pgrep exchange_server)
root@COL334 Assignment:~/COL334-A2 # sockstat -4 | grep -c ':5000'
157052
root@COL334 Assignment:~/COL334-A2 # procstat -f $PID | wc -l
   70011
root@COL334 Assignment:~/COL334-A2 # ps -o pid,rss,%cpu -p $PID
  PID   RSS %CPU
21302 26196  0.0
root@COL334 Assignment:~/COL334-A2 # sysctl kern.openfiles kern.maxfiles
kern.openfiles: 140123
kern.maxfiles: 1000000
root@COL334 Assignment:~/COL334-A2 # netstat -m
577/3743/4320 mbufs in use (current/cache/total)
0/1270/1270/262144 mbuf clusters in use (current/cache/total/max)
0/1270 mbuf+clusters out of packet secondary zone in use (current/cache)
320/1204/1524/127425 4k (page size) jumbo clusters in use (current/cache/total/max)
0/0/0/37755 9k jumbo clusters in use (current/cache/total/max)
0/0/0/21237 16k jumbo clusters in use (current/cache/total/max)
1424K/8291K/9716K bytes allocated to network (current/cache/total)
0/0/0 requests for mbufs denied (mbufs/clusters/mbuf+clusters)
0/0/0 requests for mbufs delayed (mbufs/clusters/mbuf+clusters)
0/0/0 requests for jumbo clusters delayed (4k/9k/16k)
0/0/0 requests for jumbo clusters denied (4k/9k/16k)
0 sendfile syscalls
0 sendfile syscalls completed without I/O request
0 requests for I/O initiated by sendfile
0 pages read by sendfile as part of a request
0 pages were valid at time of a sendfile request
0 pages were valid and substituted to bogus page
0 pages were requested for read ahead by applications
0 pages were read ahead by sendfile
0 times sendfile encountered an already busy page
0 requests for sfbufs denied
0 requests for sfbufs delayed
root@COL334 Assignment:~/COL334-A2 #
```

Figure 25: `poll` build holding 70 000 idle connections — `make`, server on :5000, then `procstat` / `ps` / `sysctl` / `netstat -m`

Both mechanisms sustain the 70 000-connection target with negligible CPU and zero socket-buffer pressure; the `poll` build actually uses ~18 MB *less* RSS, because its per-connection bookkeeping is just one `pollfd` plus a hash-map slot, whereas the `kqueue` `Poller` also keeps a per-fd interest-state map and kernel-side filter registrations. The `kqueue` cost that does not appear here is one extra `kevent` syscall whenever a connection's read/write interest changes — pure overhead on a small, trade-saturated deployment, which is why **`poll` is the default build**. The `kqueue` benefit that also does not appear in this *idle* test is O(ready) event servicing: it only pays off once a large idle population coexists with active traffic, where `poll` must scan all 70 000 descriptors on every wakeup and `kqueue` does not.